



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА - Российский технологический университет»
РТУ МИРЭА

Институт Информационных Технологий
Кафедра Вычислительной Техники

ПРАКТИЧЕСКАЯ РАБОТА №4

по дисциплине
«Разработка обеспечивающих подсистем систем поддержки
принятия решений»

Студент группы: ИКБО-04-22

Кликушин В.И.
(Ф. И.О. студента)

Преподаватель

Гуличева А.А.
(Ф.И.О. преподавателя)

Москва 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 ПОСТАНОВКА ЗАДАЧИ	4
2 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	5
3 ДОКУМЕНТАЦИЯ К ДАННЫМ.....	7
3.1 Описание предметной области	7
3.2 Анализ данных.....	7
3.3 Предобработка данных	10
4 ПРАКТИЧЕСКАЯ ЧАСТЬ	11
4.1 Функциональные возможности	11
4.2 Метод главных компонент	14
4.3 Построение РСА модели	15
4.4 Анализ главных компонент.....	17
4.5 Рекомендации на основе РСА.....	20
4.6 Кластеризация фильмов	21
4.7 Визуализация и интерпретация	23
ЗАКЛЮЧЕНИЕ	25
СПИСОК ИНФОРМАЦИОННЫХ ИСТОЧНИКОВ.....	26
ПРИЛОЖЕНИЯ.....	27

ВВЕДЕНИЕ

Современные стриминговые платформы и онлайн-кинотеатры сталкиваются с проблемой эффективного управления вниманием пользователей в условиях практически неограниченного выбора контента. С расширением каталогов до десятков тысяч фильмов традиционные методы ручного поиска становятся недостаточными для удовлетворения индивидуальных предпочтений. Проблема информационной перегрузки особенно остро стоит в сфере развлекательного контента, где пользователи тратят значительное время на поиск релевантных фильмов вместо непосредственного просмотра.

В настоящее время доминирующими подходами в рекомендательных системах являются методы коллаборативной фильтрации, основанные на поиске схожих пользователей или объектов, а также алгоритмы матричных разложений. Однако эти методы часто работают с избыточной размерностью данных и не всегда предоставляют интерпретируемые результаты. В отличие от них, метод главных компонент (РСА) позволяет выявлять скрытые семантические паттерны в характеристиках контента, сокращая размерность данных при минимальной потере информации.

В данной работе предлагается комплексное применение метода главных компонент для построения интерпретируемой рекомендательной системы. РСА применяется к семантически осмысленным признакам фильмов — жанрам, возрасту, рейтингам.

Для экспериментальной проверки эффективности подхода выбран датасет MovieLens, содержащий реальные оценки пользователей. В работе реализована многомодульная рекомендательная система, включающая как традиционные методы (коллаборативная фильтрация, SVD), так и модель на основе РСА, что позволяет проводить сравнительный анализ эффективности различных подходов. Особое внимание уделено визуализации и интерпретации главных компонент, кластеризации фильмов в сниженном пространстве и генерации объяснимых рекомендаций.

1 ПОСТАНОВКА ЗАДАЧИ

Цель работы: приобрести навыки реализации модельных коллаборативных методов рекомендательных систем, основанных на поиске скрытых факторов методом главных компонент.

Задачи: создать программную реализацию модельного метода рекомендательной системы (РС), основанную на поиске главных компонент, включающую актуальную предметную область для применения РС (вроде маркетплейса, медиа ресурсов, соц. сетей, экономической сферы и т. д.) и набор начальных данных для неё (опрос покупателей, статистка за временной период и т.д., включающий набор признаков, наиболее важных и наиболее коррелирующих друг с другом. По возможности отцентрировать выборку (для минимизации потерь при дальнейших расчётах, так как при таком сдвиге средние значения признаков будут стремиться к нулю), при большой разнице порядков признаков необходимо выполнить их стандартизацию или масштабирование, т. к. дисперсия величин может получить слишком большой или слишком малый размер, из-за чего понижение размерности и проецирование может получиться некорректным и иметь слишком малую точность при восстановлении первоначальных данных. На основе модифицированной выборки построить ковариационную матрицу признаков, найти её собственные векторы и рассчитать её собственные значения, определив новый базис для начальной выборки. Выполнить снижение размерности по новому базису для начальных характеристик путём проекции на каждый из полученных собственных векторов, с помощью восстановления начальных данных и проверки полученного результата восстановления с настоящими начальными данными выбрать вектор, проекция на который имеет наименьшую функцию потерь.

2 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Метод главных компонент — это статистический метод линейного преобразования данных, позволяющий уменьшать размерность признакового пространства при минимальной потере информации. Основная идея РСА состоит в поиске новой ортогональной системы координат, в которой первые компоненты объясняют максимальную дисперсию исходных данных. Это достигается путём анализа структуры ковариаций между признаками и вычисления собственных векторов соответствующей матрицы.

Пусть имеется матрица данных (Формула 2.1).

$$X \in R^{n \times d}, \quad (2.1)$$

где n — число объектов;

d — оценка, поставленная пользователем u объекту i .

Так как РСА чувствителен к масштабу данных, перед вычислениями признаки стандартизируют. Сначала выполняется центрирование (Формула 2.2).

$$\overline{X}_{ij} = X_{ij} - \mu_j, \quad (2.2)$$

где μ_j — среднее значение признака j ;

X_{ij} — значение j -го признака у объекта i .

Если признаки имеют разные масштабы, данные нормируются (Формула 2.3).

$$\overline{X}_{ij} = \frac{\overline{X}_{ij}}{\sigma_j}, \quad (2.3)$$

где σ_j — стандартное отклонение признака j .

После стандартизации вычисляется ковариационная матрица признаков

(Формула 2.4).

$$C = \frac{1}{n-1} \bar{X}^T \bar{X} \quad (2.4)$$

Элемент C_{ij} показывает, насколько линейно связаны признаки i и j . Если признаки скоррелированы, ковариационная матрица содержит значимые внедиагональные элементы, и именно эта структура позволяет РСА находить «направления наибольшей изменчивости данных».

Главные компоненты являются собственными векторами ковариационной матрицы C , которые решают уравнение, представленное Формулой 2.5.

$$C v_k = \lambda_k v_k, \quad (2.5)$$

где λ_k – собственное значение (дисперсия вдоль этого направления);

v_k – собственный вектор (направление k -й главной компоненты).

Собственные значения упорядочивают по убыванию: $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$.

Первое направление v_1 — это направление максимальной дисперсии, второе — максимальной дисперсии при условии ортогональности к первому, и так далее.

На практике РСА вычисляют не через собственные значения, а через сингулярное разложение матрицы $\bar{X}$.

3 ДОКУМЕНТАЦИЯ К ДАННЫМ

3.1 Описание предметной области

Данные взяты из открытого сервиса MovieLens, предоставляемого исследовательской группой GroupLens Университета Миннесоты. Датасет ml-latest-small содержит информацию о рейтингах и тегах пользователей для фильмов.

Целью использования этих данных является построение рекомендательной системы фильмов, которая может предсказывать оценки для фильмов, не просмотренных пользователем, и рекомендовать новые фильмы на основе исторических оценок и тегов.

Датасет состоит из следующих сущностей:

- пользователи (User): анонимизированные идентификаторы пользователей, которые выставляли оценки фильмов и добавляли теги;
- фильмы (Movie): идентификаторы, название, жанры, год выпуска и ссылки на внешние источники (IMDb, TMDb);
- рейтинги (Rating): пользователь, фильм, оценка от 0.5 до 5 и временная метка;
- теги (Tag): пользователь, фильм, текст тега и временная метка.

3.2 Анализ данных

Датасет ml-latest-small представляет собой компактную выборку данных MovieLens, содержащую оценки пользователей и информацию о фильмах. Он включает чуть более ста тысяч рейтингов, чуть меньше десяти тысяч фильмов и около шести сотен пользователей, что делает его удобным для экспериментов и обучения рекомендательных систем без необходимости использования мощных вычислительных ресурсов.

Каждая запись о рейтинге состоит из идентификатора пользователя, идентификатора фильма, выставленной оценки и временной метки. Основная информация о данных в файле ratings.csv представлена на Рисунке 3.2.1.

```
><class 'pandas.core.frame.DataFrame'>
RangeIndex: 100836 entries, 0 to 100835
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   userId      100836 non-null  int64
1   movieId     100836 non-null  int64
2   rating       100836 non-null  float64
3   timestamp   100836 non-null  int64
dtypes: float64(1), int64(3)
memory usage: 3.1 MB
```

Рисунок 3.2.1 - Основная информация о данных в файле ratings.csv

Распределение выставленных оценок представлено на Рисунке 3.2.2.

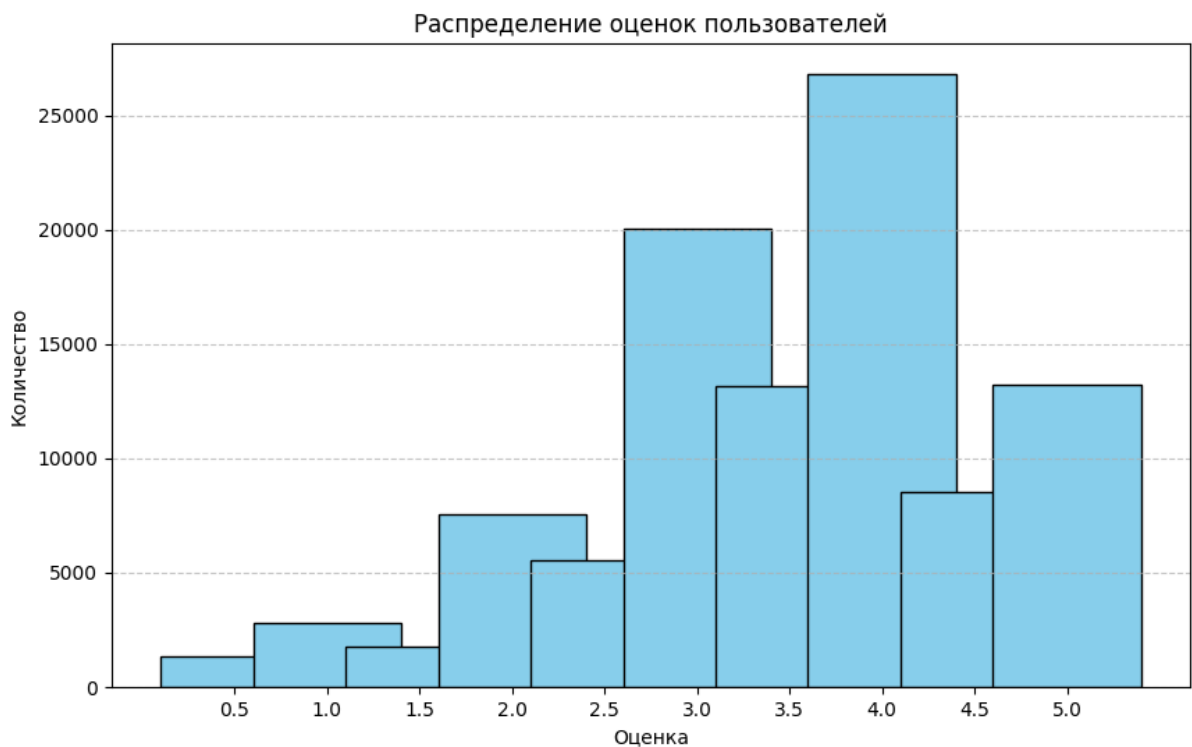


Рисунок 3.2.2 – Распределение оценок

Оценки распределены по шкале от 0.5 до 5 с шагом 0.5, что позволяет достаточно точно моделировать предпочтения пользователей, одновременно упрощая обработку данных. Анализ распределения оценок показывает, что наиболее популярные значения находятся в диапазоне от 3 до 4 баллов, что отражает склонность пользователей давать средние и положительные оценки.

Более экстремальные оценки, например 0.5 или 5, встречаются реже, но именно они могут быть особенно информативны при построении рекомендательных моделей, так как ясно отражают сильные предпочтения или отторжение.

Фильмы в датасете сопровождаются информацией о жанрах. Основная информация о данных в файле movies.csv отображена на Рисунке 3.2.3.

```
>>> df
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9742 entries, 0 to 9741
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   movieId     9742 non-null   int64
1   title       9742 non-null   object
2   genres      9742 non-null   object
dtypes: int64(1), object(2)
memory usage: 228.5+ KB
```

Рисунок 3.2.3 - Основная информация о данных в файле movies.csv

Датафрейм не содержит нулевых значений или пропусков. Первые 5 строк датафрейма представлены на Рисунке 3.2.4.

	movieId	title	genres
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
1	2	Jumanji (1995)	Adventure Children Fantasy
2	3	Grumpier Old Men (1995)	Comedy Romance
3	4	Waiting to Exhale (1995)	Comedy Drama Romance
4	5	Father of the Bride Part II (1995)	Comedy

(.venv) PS C:\python_projects\MIREA\Разработка обеспечивающих подсистем систем поддержки приня

Рисунок 3.2.4 – Первые пять строк датафрейма movies

Каждый фильм может относиться к нескольким жанрам, которые объединены в одну строку через символ «|». Анализ жанров показывает, что большинство фильмов имеют два-три жанра, что отражает реальную практику киноиндустрии, где произведения редко ограничиваются одним жанром. Также есть небольшое количество фильмов без указанных жанров, что требует дополнительной предобработки для их корректного включения в рекомендательную систему.

В названиях фильмов часто содержится год выпуска, заключённый в скобки. Это позволяет извлечь дополнительную информацию о времени выхода фильма и анализировать данные по временным периодам.

Теги, оставленные пользователями, представляют собой текстовые описания, которые дают дополнительные сведения о предпочтениях.

Информация о данных в файле tags.csv представлена на Рисунке 3.2.5.

```
><class 'pandas.core.frame.DataFrame'>
RangeIndex: 3683 entries, 0 to 3682
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   userId      3683 non-null   int64
1   movieId     3683 non-null   int64
2   tag         3683 non-null   object
3   timestamp   3683 non-null   int64
dtypes: int64(3), object(1)
memory usage: 115.2+ KB
```

Рисунок 3.2.5 - Основная информация о данных в файле tags.csv

Анализ тегов показывает, что пользователи используют их довольно разнообразно: одни ограничиваются жанровыми метками, другие оставляют более субъективные описания вроде «funny» или «thriller». Теги помогают расширить рекомендации, особенно в гибридных системах, где учитываются как оценки, так и семантические характеристики фильмов.

Временные метки, присутствующие как в рейтингах, так и в тегах, позволяют анализировать динамику активности пользователей. С их помощью можно определить, когда пользователи наиболее активно выставляли оценки, выявить сезонные колебания интереса к фильмам или построить модели, учитывающие эволюцию предпочтений во времени.

Общий анализ показывает, что датасет ml-latest-small хорошо сбалансирован для экспериментов с рекомендательными системами. Он содержит достаточно данных для выявления закономерностей в поведении пользователей и предпочтениях фильмов.

3.3 Предобработка данных

В рамках предобработки удален год выпуска фильма из названия и добавлен отдельный столбец year в соответствующий датафрейм.

Временные метки приведены к типу данных datetime для более удобной обработки. Жанры преобразованы в список, обработаны фильмы с отсутствующими жанрами.

4 ПРАКТИЧЕСКАЯ ЧАСТЬ

4.1 Функциональные возможности

В практической части реализована интерактивная консольная система для работы с данными MovieLens. Основная цель заключалась в создании среды, позволяющей пользователю просматривать фильмы, искать их по названию и жанрам, оценивать и получать рекомендации на основе собственных оценок и оценок других пользователей. Полный код реализации кинотеатра с рекомендательной системой представлен в Приложении А.

Главное меню кинотеатра представлено на Рисунке 4.1.1.

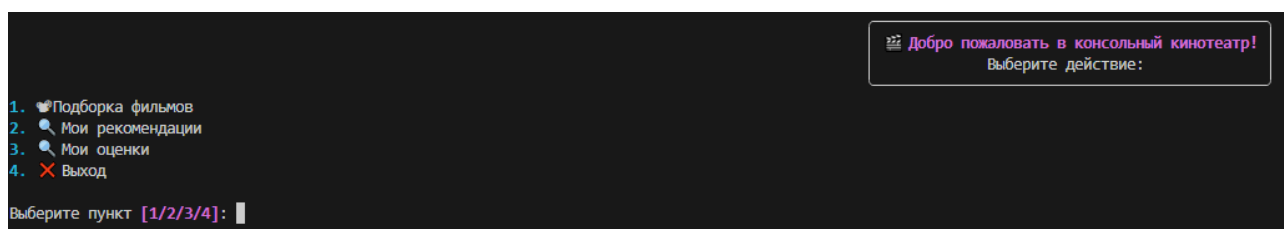


Рисунок 4.1.1 – Главное меню

Пользователю доступен просмотр каталога фильмов, ленты рекомендаций, ранее поставленных оценок, также реализована возможность выхода из приложения.

Каталог фильмов представлен на Рисунке 4.1.2.

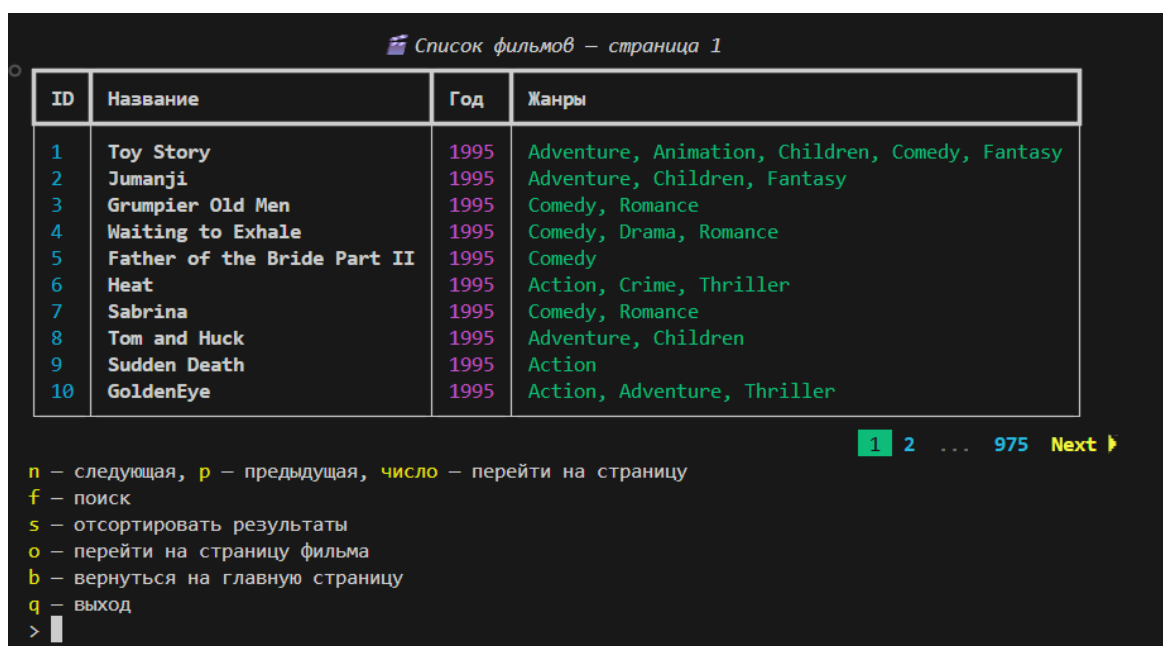


Рисунок 4.1.2 – Каталог фильмов

Сортировка каталога возможна по году выпуска фильм, по жанрам, по идентификатору. Пример сортировки каталога представлен на Рисунке 4.1.3.

Сортировка по Год (desc) – страница 1

ID	Название	Год	Жанры
183635	Maze Runner: The Death Cure	2018	Action, Mystery, Sci-Fi, Thriller
183959	Tom Segura: Disgraceful	2018	Comedy
185033	I Kill Giants	2018	Drama, Fantasy, Thriller
188797	Tag	2018	Comedy
185473	Blockers	2018	Comedy
185435	Game Over, Man!	2018	Action, Comedy
183611	Game Night	2018	Action, Comedy, Crime, Horror
185585	Pacific Rim: Uprising	2018	Action, Fantasy, Sci-Fi
184471	Tomb Raider	2018	Action, Adventure, Fantasy
183295	Insidious: The Last Key	2018	Horror, Mystery, Thriller

1 2 ... 975 Next ▶

n – следующая, p – предыдущая, число – перейти на страницу
o – открыть страницу фильма, b – назад
> █

Рисунок 4.1.3 – Сортировка каталога по году по убыванию

Добавлен механизм поиска по каталогу с фильтрацией по году выпуска, жанрам. Поиск осуществляется по названию путем проверки заданной подстроки. Интерфейс поиска представлен на Рисунке 4.1.4.

Введите название фильма (Enter – пропустить): alien

Выберите жанры

№	Жанр
1	Action
2	Adventure
3	Animation
4	Children's
5	Comedy
6	Crime
7	Documentary
8	Drama
9	Fantasy
10	Film-Noir
11	Horror
12	Musical
13	Mystery
14	Romance
15	Sci-Fi
16	Thriller
17	War
18	Western

Введите номера жанров через пробел (Enter – пропустить): █

Рисунок 4.1.4 – Интерфейс поиска

Результат поиска на запрос alien представлены на Рисунке 4.1.5.

Результаты поиска: 20

ID	Название	Год	Жанры
1200	Aliens	1986	Action, Adventure, Horror, Sci-Fi
1214	Alien	1979	Horror, Sci-Fi
1320	Alien³ (a.k.a. Alien 3)	1992	Action, Horror, Sci-Fi, Thriller
1690	Alien: Resurrection	1997	Action, Horror, Sci-Fi
3701	Alien Nation	1988	Crime, Drama, Sci-Fi, Thriller
4526	My Stepmother Is an Alien	1988	Comedy, Romance, Sci-Fi
5051	Italian for Beginners (Italiensk for begyndere)	2000	Comedy, Drama, Romance
5704	Without Warning (a.k.a. Alien Warning) (a.k.a. It Came Without Warning)	1980	Horror, Sci-Fi
6835	Alien Contamination	1980	Action, Horror, Sci-Fi
6899	Alien from L.A.	1988	Sci-Fi

1 2 Next ▶

Фильтры: Название: alien, Жанры: —, Год: —
 n — следующая, p — предыдущая, число — перейти на страницу
 s — отсортировать результаты
 o — открыть страницу фильма по ID
 r — сброс фильтров
 b — назад
 > |

Рисунок 4.1.5 – Результаты поиска на запрос alien

Для составлений рекомендаций пользователю необходимо поставить оценки фильмам. Для этого реализована возможность открытия «страницы» фильма, где доступен просмотр тэгов фильма и присутствует возможность оценить фильм.

Интерфейс страницы фильма представлен на Рисунке 4.1.6.

ID 1200

 Название: Aliens
  Год: 1986
  Жанры: ['Action', 'Adventure', 'Horror', 'Sci-Fi']

Теги

action, aliens, horror, sci-fi, space, space craft, SPACE TRAVEL, suspense, space

Доступные действия:

r — оценить фильм
 b — назад
 q — выйти из приложения
 > |

Рисунок 4.1.6 – Интерфейс страницы фильма

Пример выставления оценки показан на Рисунке 4.1.7.

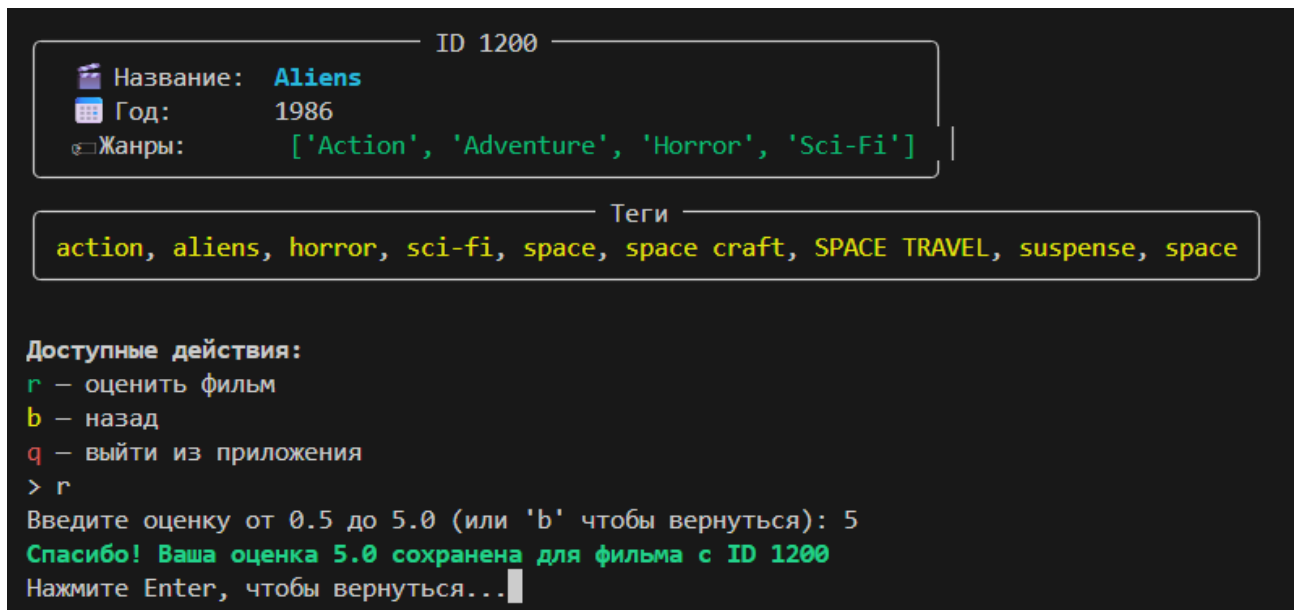


Рисунок 4.1.7 – Выставление оценки для фильма

Проставленные оценки можно посмотреть на отдельной странице в главном меню (Рисунок 4.1.8).

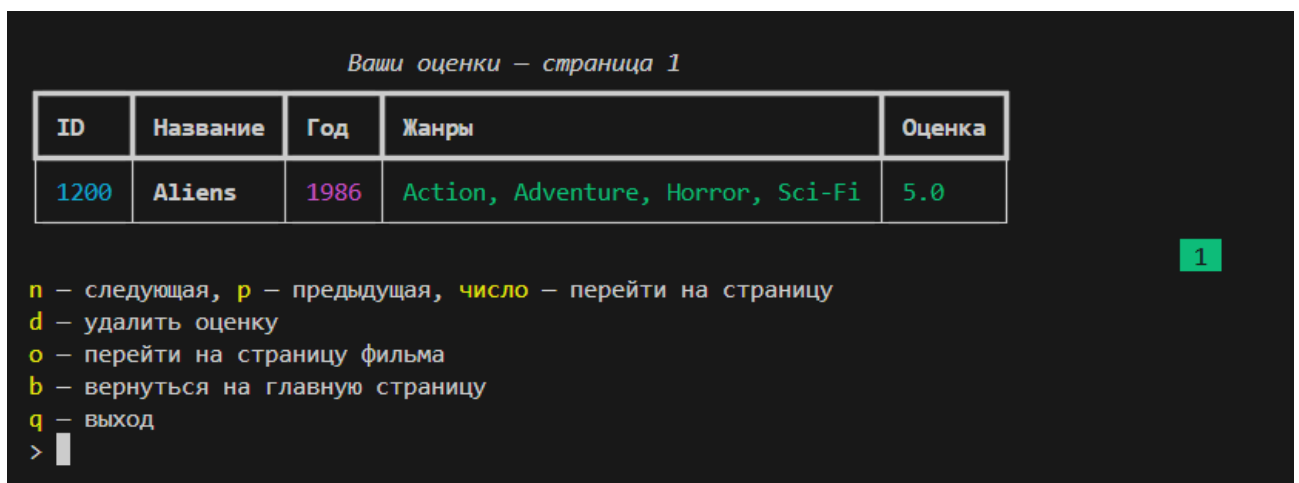


Рисунок 4.1.8 – Проставленные оценки

Добавлена возможность удалить оценку по идентификатору фильма (Рисунок 4.1.9).

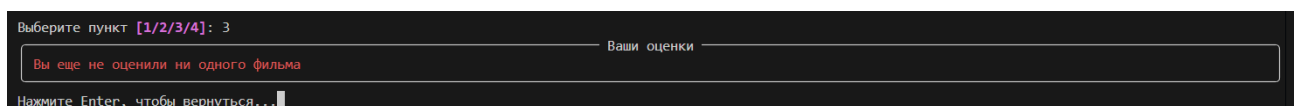


Рисунок 4.1.9 – Удаление оценки

Оценки на фильм с идентификатором 1200 успешно удалена, текущий пользователь не оценил ни одного фильма.

4.2 Метод главных компонент

В системе реализован комплексный анализ главных компонент (PCA) как альтернативный подход к рекомендациям фильмов. В отличие от SVD, работающего с матрицей оценок пользователей, PCA применяется к семантически осмысленной матрице признаков фильмов, что позволяет выявлять скрытые закономерности в характеристиках кинопроизведений. Код файла PCARecommdender.py представлен в Приложении Д.

Главное меню системы включает раздел «Анализ главных компонент (PCA)», предоставляющий доступ к следующим функциям:

- построение PCA модели с автоматическим или ручным выбором числа компонент;
- рекомендация похожих фильмов на основе семантического сходства;
- детальный анализ компонент с интерпретацией скрытых факторов;
- визуализация объясненной дисперсии компонент;
- кластеризация фильмов в пространстве главных компонент;
- персональные рекомендации на основе PCA;
- объяснение рекомендаций с семантической интерпретацией.

Реализованный раздел меню «Анализ главных компонент» представлен на Рисунке 4.2.1.

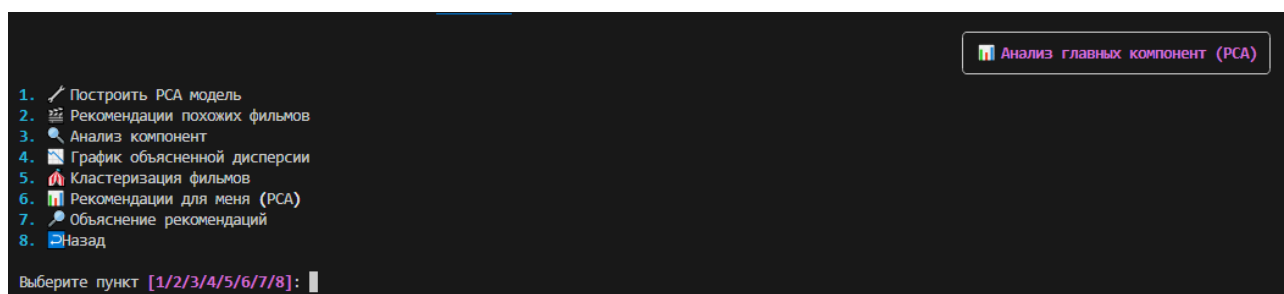


Рисунок 4.2.1 - Реализованный раздел меню «Анализ главных компонент»

4.3 Построение PCA модели

Модель главных компонент строится на основе комплексной матрицы признаков фильмов, включающей:

- бинарные признаки для каждого жанра, отражающие принадлежность фильма к соответствующим категориям;

- возраст фильма в годах;
- индикаторы старых и новых релизов;
- нормализованные значения возраста;
- средний рейтинг фильма;
- стандартное отклонение оценок;
- количество оценок;
- нормализованные версии статистических показателей.

Матрица признаков стандартизируется с использованием StandardScaler перед применением PCA. Система поддерживает как автоматический выбор числа компонент (на основе объясненной дисперсии), так и ручное задание количества главных компонент.

Качество построенной модели оценивается через метрики восстановления исходных данных: MSE, RMSE, MAE.

Построение модели PCA с автоматическим выбором числа компонент представлено на Рисунке 4.3.1.

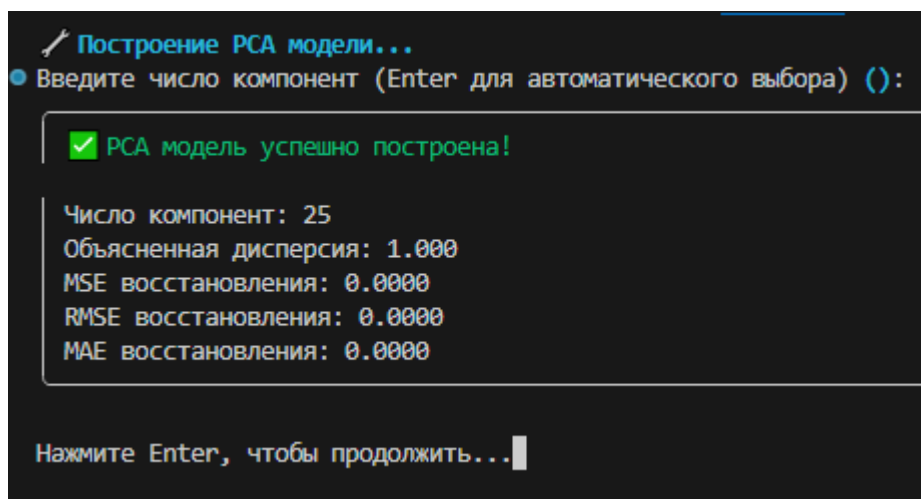


Рисунок 4.3.1 - Построение модели PCA с автоматическим выбором числа компонент

Результаты демонстрируют успешное построение модели с 25 главными компонентами, при этом наблюдается полное объяснение дисперсии исходных данных и нулевые ошибки восстановления.

Объясненная дисперсия, равная единице, свидетельствует о том, что все 25 главных компонент в совокупности полностью сохраняют информацию исходного признакового пространства. Это характерная ситуация, когда

количество выделенных компонент совпадает с рангом исходной матрицы признаков. В контексте задачи рекомендаций фильмов это означает, что семантическое пространство, построенное на основе жанровых, временных и рейтинговых характеристик, полностью описывается 25 ортогональными направлениями максимальной дисперсии.

Построение модели PCA с заданным числом компонент отображено на Рисунке 4.3.2.

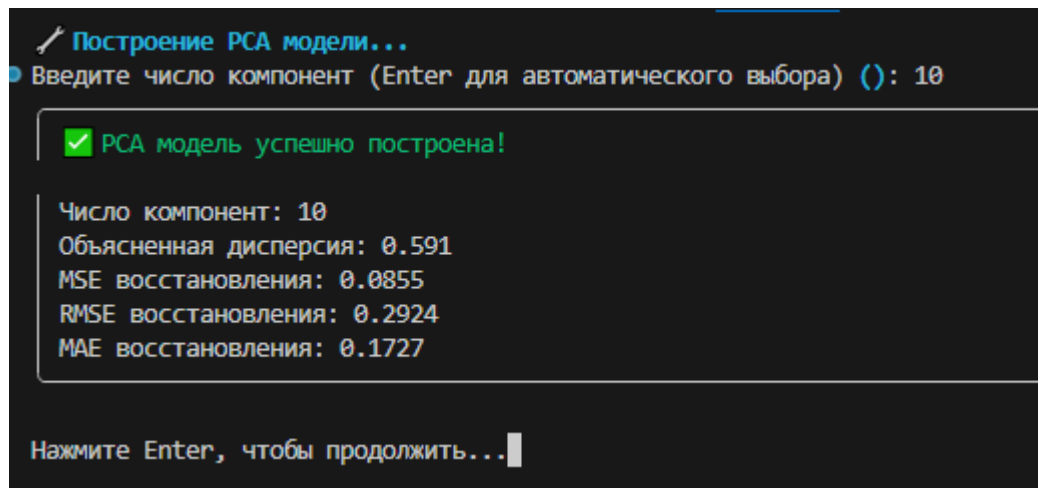


Рисунок 4.3.2 - Построение модели PCA с заданным числом компонент

Данный результат демонстрирует работу PCA-модели, сократившей размерность данных до 10 компонент, которые объясняют 59.1% исходной дисперсии. Полученные ошибки восстановления указывают на умеренную потерю информации, что является ожидаемым компромиссом при снижении размерности признакового пространства.

4.4 Анализ главных компонент

Система предоставляет инструменты для детального анализа полученных главных компонент. Для каждой компоненты вычисляется:

- вклад признаков: определяются признаки, оказывающие наибольшее влияние на компоненту, с указанием направления влияния (положительного или отрицательного);
- интерпретация компонент: автоматическая семантическая

интерпретация скрытых факторов на основе наиболее значимых признаков;

- объясненная дисперсия: для каждой компоненты вычисляется процент дисперсии исходных данных, который она объясняет.

Интерфейс анализа компонент позволяет пользователю исследовать каждую компоненту отдельно, получая таблицу с ранжированными по важности признаками и их интерпретацию в понятных терминах. На Рисунке 4.4.1 представлен детальный анализ первой главной компоненты PCA-модели, демонстрирующий ключевые аспекты методологии интерпретации скрытых факторов.

Введите номер компоненты (1-10) (1): 1
Анализ 1-й главной компоненты

Признак	Вклад	Абсолютное значение
genre_Adventure	0.3871	0.3871
genre_Animation	0.3143	0.3143
genre_Children	0.3074	0.3074
movie_age_norm	-0.3016	0.3016
genre_Drama	-0.2985	0.2985
genre_Fantasy	0.2844	0.2844
genre_Action	0.2822	0.2822
is_old	-0.2558	0.2558
genre_Sci-Fi	0.2508	0.2508
genre_IMAX	0.2315	0.2315
rating_std_norm	0.1613	0.1613
rating_count_norm	0.1567	0.1567
genre_Romance	-0.1473	0.1473
is_recent	0.1299	0.1299
rating_mean_norm	-0.1161	0.1161
genre_Film-Noir	-0.1089	0.1089
genre_War	-0.0892	0.0892
genre_Crime	-0.0686	0.0686
genre_Western	-0.0391	0.0391
genre_Documentary	-0.0383	0.0383

Интерпретация компоненты 1:

- Положительно связана с: жанр 'Adventure', жанр 'Animation', жанр 'Children'
- Отрицательно связана с: жанр 'Drama'

Объясненная дисперсия: 9.18%

Нажмите Enter, чтобы продолжить...

Рисунок 4.4.1 – Пример проанализированной первой компоненты

Первая компонента объясняет 9.18% общей дисперсии данных и выявляет четкий семантический паттерн: положительная корреляция с жанрами «Adventure», «Animation», «Children» и отрицательная - с жанром «Drama». Это позволяет интерпретировать данную компоненту как ось «семейные развлекательные фильмы против серьезных драматических произведений».

Компоненты вычисляются последовательно: первая компонента захватывает максимально возможную дисперсию данных, вторая - максимальную дисперсию из оставшейся при условии ортогональности первой, и так далее. Такой подход обеспечивает оптимальное сжатие информации, где первые несколько компонент содержат наиболее значимые закономерности. Пример проанализированной второй компоненты представлен на Рисунке 4.4.2.

Введите номер компоненты (1-10) (1): 2
Анализ 2-й главной компоненты

Признак	Вклад	Абсолютное значение
genre_Thriller	0.4541	0.4541
genre_Children	-0.3187	0.3187
genre_Comedy	-0.2917	0.2917
movie_age_norm	-0.2830	0.2830
genre_Crime	0.2671	0.2671
is_old	-0.2661	0.2661
genre_Animation	-0.2489	0.2489
genre_Musical	-0.2480	0.2480
genre_Mystery	0.2275	0.2275
genre_Action	0.2194	0.2194
genre_Horror	0.2106	0.2106
genre_Romance	-0.1805	0.1805
genre_Fantasy	-0.1585	0.1585
genre_Sci-Fi	0.1265	0.1265
genre_Adventure	-0.1186	0.1186
is_recent	0.1015	0.1015
genre_Drama	0.0785	0.0785
rating_mean_norm	-0.0770	0.0770
genre_Western	-0.0549	0.0549
genre_Film-Noir	0.0459	0.0459

Интерпретация компоненты 2:

- Положительно связана с: жанр 'Thriller', жанр 'Crime'
- Отрицательно связана с: жанр 'Children', жанр 'Comedy'

Объясненная дисперсия: 8.44%

Нажмите Enter, чтобы продолжить...

Рисунок 4.4.2 - Пример проанализированной второй компоненты

Вторая компонента объясняет меньшую дисперсию (8.44%) по сравнению с первой, что указывает на её второстепенную роль в описании данных. Она противопоставляет напряжённые и мрачные жанры (триллеры, криминал) — лёгким и детским (детское кино, комедии).

4.5 Рекомендации на основе PCA

Система реализует два типа рекомендаций на основе метода главных компонент.

Рекомендации похожих фильмов: поиск семантически близких фильмов в пространстве главных компонент с использованием косинусной меры сходства. Пользователь может указать фильм-образец и получить список наиболее похожих произведений с указанием степени сходства.

Пример рекомендации похожих фильмов представлен на Рисунке 4.5.1.

Введите ID фильма для поиска похожих: 1200
Количество рекомендаций (10): 20

Фильмы, похожие на 'Aliens'

ID	Название	Сходство	Жанры
2916	Total Recall	0.950	Action, Adventure, Sci-Fi, Thriller
1374	Star Trek II: The Wrath of Khan	0.945	Action, Adventure, Sci-Fi, Thriller
316	Stargate	0.942	Action, Adventure, Sci-Fi
3527	Predator	0.939	Action, Sci-Fi, Thriller
1240	Terminator, The	0.939	Action, Sci-Fi, Thriller
541	Blade Runner	0.938	Action, Sci-Fi, Thriller
1127	Abyss, The	0.937	Action, Adventure, Sci-Fi, Thriller
442	Demolition Man	0.935	Action, Adventure, Sci-Fi
3703	Road Warrior, The (Mad Max 2)	0.917	Action, Adventure, Sci-Fi, Thriller
1210	Star Wars: Episode VI - Return of the Jedi	0.917	Action, Adventure, Sci-Fi
1371	Star Trek: The Motion Picture	0.912	Adventure, Sci-Fi
2641	Superman II	0.912	Action, Sci-Fi
1129	Escape from New York	0.912	Action, Adventure, Sci-Fi, Thriller
480	Jurassic Park	0.912	Action, Adventure, Sci-Fi, Thriller
2640	Superman	0.911	Action, Adventure, Sci-Fi
1196	Star Wars: Episode V - The Empire Strikes Back	0.911	Action, Adventure, Sci-Fi
2021	Dune	0.911	Adventure, Sci-Fi
208	Waterworld	0.904	Action, Adventure, Sci-Fi
1356	Star Trek: First Contact	0.902	Action, Adventure, Sci-Fi, Thriller
3698	Running Man, The	0.900	Action, Sci-Fi

Характеристики исходного фильма:
Фильм характеризуется: жанр 'Action', жанр 'Adventure', жанр 'Horror', жанр 'Sci-Fi'

Нажмите Enter, чтобы продолжить...

Рисунок 4.5.1 - Пример рекомендации похожих фильмов

Персональные рекомендации: генерация рекомендаций на основе оценок текущего пользователя. Система строит профиль пользователя в пространстве главных компонент как взвешенную комбинацию векторов оцененных фильмов, затем находит наиболее близкие к этому профилю неоцененные фильмы.

Персональные рекомендации отображены на Рисунке 4.5.2.

PCA-модель формируется...

Качество восстановления:
MSE: 0.0855
RMSE: 0.2924
MAE: 0.1727

Количество рекомендаций (20):

PCA рекомендации

ID	Название	Схожесть	Жанры	Объяснение
2916	Total Recall	0.950	Action, Adventure, Sci-Fi, Thriller	Фильм характеризуется: жанр 'Action', жанр 'Advent...
1374	Star Trek II: The Wrath of Khan	0.945	Action, Adventure, Sci-Fi, Thriller	Фильм характеризуется: жанр 'Action', жанр 'Advent...
316	Stargate	0.942	Action, Adventure, Sci-Fi	Фильм характеризуется: жанр 'Action', жанр 'Advent...
3527	Predator	0.939	Action, Sci-Fi, Thriller	Фильм характеризуется: жанр 'Action', жанр 'Sci-Fi...
1240	Terminator, The	0.939	Action, Sci-Fi, Thriller	Фильм характеризуется: жанр 'Action', жанр 'Sci-Fi...
541	Blade Runner	0.938	Action, Sci-Fi, Thriller	Фильм характеризуется: жанр 'Action', жанр 'Sci-Fi...
1127	Abyss, The	0.937	Action, Adventure, Sci-Fi, Thriller	Фильм характеризуется: жанр 'Action', жанр 'Advent...
442	Demolition Man	0.935	Action, Adventure, Sci-Fi	Фильм характеризуется: жанр 'Action', жанр 'Advent...
3703	Road Warrior, The (Mad Max 2)	0.917	Action, Adventure, Sci-Fi, Thriller	Фильм характеризуется: жанр 'Action', жанр 'Advent...
1210	Star Wars: Episode VI - Return of the Jedi	0.917	Action, Adventure, Sci-Fi	Фильм характеризуется: высоким средним рейтингом, ...
1371	Star Trek: The Motion Picture	0.912	Adventure, Sci-Fi	Фильм характеризуется: жанр 'Adventure', жанр 'Sci...
2641	Superman II	0.912	Action, Sci-Fi	Фильм характеризуется: жанр 'Action', жанр 'Sci-Fi...
1129	Escape from New York	0.912	Action, Adventure, Sci-Fi, Thriller	Фильм характеризуется: жанр 'Action', жанр 'Advent...
480	Jurassic Park	0.912	Action, Adventure, Sci-Fi, Thriller	Фильм характеризуется: жанр 'Action', жанр 'Advent...
2640	Superman	0.911	Action, Adventure, Sci-Fi	Фильм характеризуется: жанр 'Action', жанр 'Advent...
1196	Star Wars: Episode V - The Empire Strikes Back	0.911	Action, Adventure, Sci-Fi	Фильм характеризуется: высоким средним рейтингом, ...
2021	Dune	0.911	Adventure, Sci-Fi	Фильм характеризуется: жанр 'Adventure', жанр 'Sci...
208	Waterworld	0.904	Action, Adventure, Sci-Fi	Фильм характеризуется: жанр 'Action', жанр 'Advent...
1356	Star Trek: First Contact	0.902	Action, Adventure, Sci-Fi, Thriller	Фильм характеризуется: жанр 'Action', жанр 'Advent...
3698	Running Man, The	0.900	Action, Sci-Fi	Фильм характеризуется: жанр 'Action', жанр 'Sci-Fi...

Показать подробную информацию о рекомендациях? [y/n]:

Рисунок 4.5.2 – Персональные рекомендации пользователя

Особенностью PCA-рекомендаций является возможность семантического объяснения: для каждого рекомендуемого фильма система генерирует текстовое описание, объясняющее, почему именно этот фильм был рекомендован на основе его характеристик.

4.6 Кластеризация фильмов

В пространстве главных компонент реализована кластеризация фильмов с использованием двух методов:

- агломеративная кластеризация: иерархический метод, объединяющий фильмы в кластеры на основе близости в пространстве признаков;
- модель гауссовской смеси: вероятностный метод, предполагающий, что данные порождены смесью нескольких гауссовых распределений.

Агломеративная кластеризация исходных фильмов представлена на Рисунке 4.6.1.

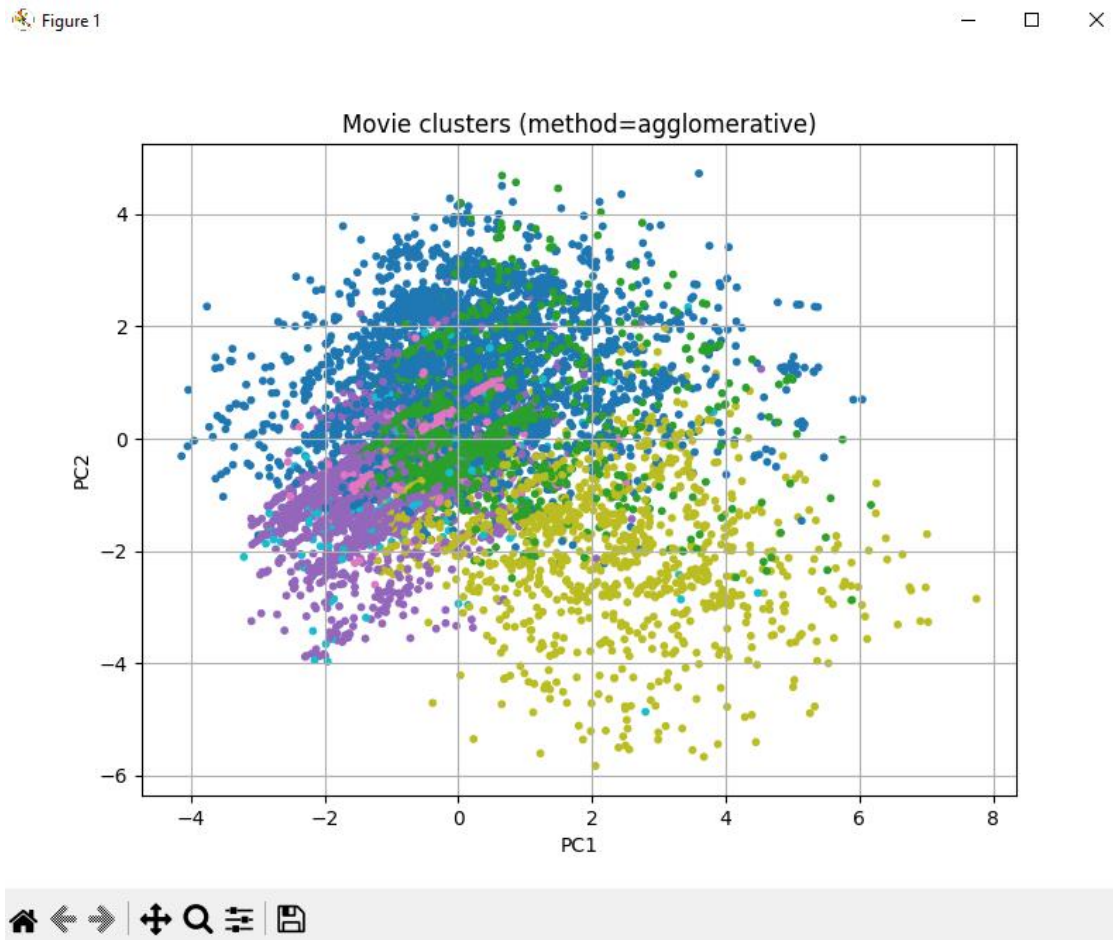


Рисунок 4.6.1 - Агломеративная кластеризация исходных фильмов

Данные группируются в три основных кластера, что подтверждает полученное распределение кластеров (Рисунок 4.6.2).

Выберите пункт [1/2/3/4/5/6/7/8]: 5
 Число кластеров (6):
 Метод кластеризации [agglomerative/gmm] (agglomerative):
 Кластеризация завершена!
 Распределение по кластерам

Кластер	Количество фильмов
0	3186
1	2832
2	2076
3	438
4	1044
5	166

Нажмите Enter, чтобы продолжить...

Рисунок 4.6.2 – Полученное распределение по кластерам при агломеративной кластеризации

Кластеризация позволяет выявлять тематические группы фильмов, не задаваясь заранее их семантикой. Визуализация кластеров в пространстве первых двух главных компонент предоставляет интуитивно понятное представление о структуре каталога.

4.7 Визуализация и интерпретация

Система предоставляет комплексные инструменты визуализации и интерпретации результатов анализа главных компонент, обеспечивая прозрачность и понятность рекомендательного процесса. Центральным элементом визуализации является график кумулятивной объясненной дисперсии, который отображает зависимость доли объясненной дисперсии от количества используемых главных компонент. Этот график служит практическим инструментом для определения оптимальной размерности пространства признаков, позволяя находить баланс между точностью представления данных и сложностью модели. Построенный график представлен на Рисунке 4.7.1.

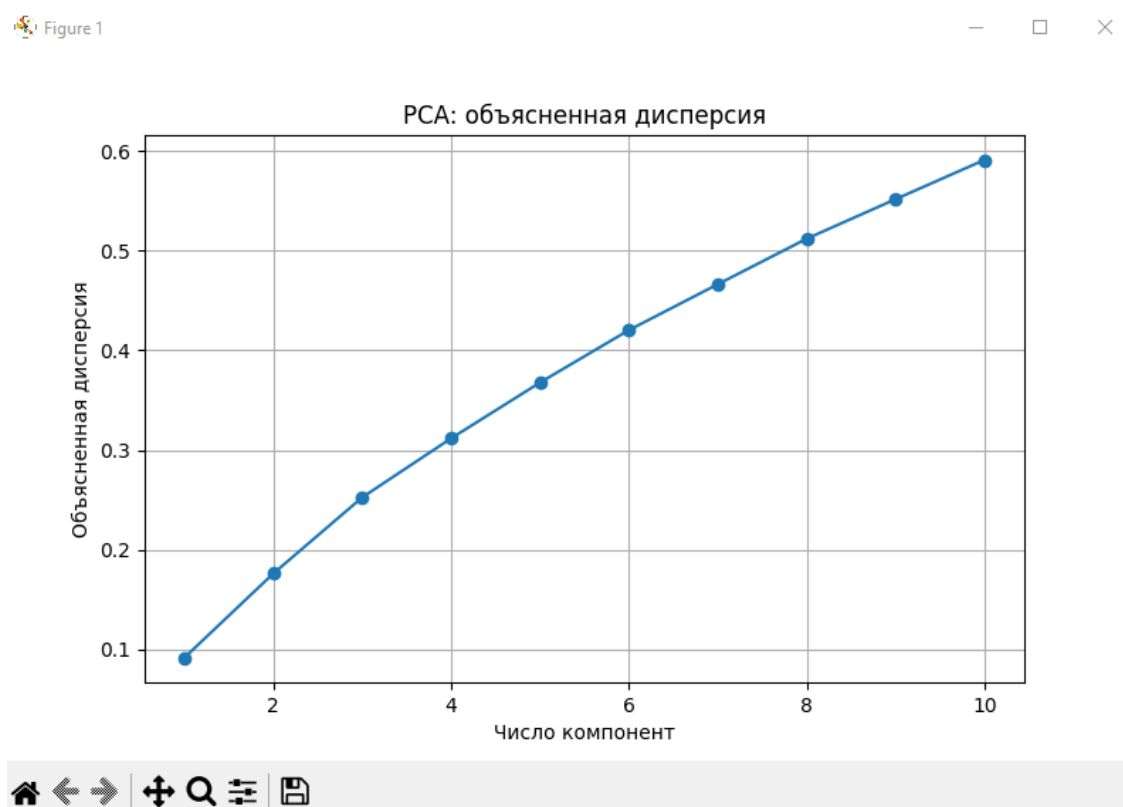


Рисунок 4.7.1 – График кумулятивной объясненной дисперсии

По Рисунку 4.7.1 видно, что первая главная компонента объясняет большую часть дисперсии данных. С добавлением последующих компонент кумулятивная объясненная дисперсия растет, но каждая следующая компонента вносит все меньший вклад.

Важной особенностью системы является возможность семантической интерпретации рекомендаций. Для каждого рекомендуемого фильма генерируется развернутое текстовое объяснение, описывающее ключевые характеристики, которые обусловили включение фильма в список рекомендаций. Объяснение формулируется на естественном языке и включает такие аспекты как жанровая принадлежность, временные характеристики фильма, статистические показатели популярности и качества. Пример интерпретации полученных рекомендаций представлен на Рисунке 4.7.2.

```
Показать подробную информацию о рекомендациях? [y/n]: y

Total Recall
Схожесть: 0.950
Объяснение: Фильм характеризуется: жанр 'Action', жанр 'Adventure', жанр 'Sci-Fi', жанр 'Thriller'

Star Trek II: The Wrath of Khan
Схожесть: 0.945
Объяснение: Фильм характеризуется: жанр 'Action', жанр 'Adventure', жанр 'Sci-Fi', жанр 'Thriller'

Stargate
Схожесть: 0.942
Объяснение: Фильм характеризуется: жанр 'Action', жанр 'Adventure', жанр 'Sci-Fi', старым релизом

Predator
Схожесть: 0.939
Объяснение: Фильм характеризуется: жанр 'Action', жанр 'Sci-Fi', жанр 'Thriller', старым релизом

Terminator, The
Схожесть: 0.939
Объяснение: Фильм характеризуется: жанр 'Action', жанр 'Sci-Fi', жанр 'Thriller', старым релизом

Blade Runner
Схожесть: 0.938
Объяснение: Фильм характеризуется: жанр 'Action', жанр 'Sci-Fi', жанр 'Thriller', старым релизом

Abyss, The
Схожесть: 0.937
Объяснение: Фильм характеризуется: жанр 'Action', жанр 'Adventure', жанр 'Sci-Fi', жанр 'Thriller'

Demolition Man
Схожесть: 0.935
Объяснение: Фильм характеризуется: жанр 'Action', жанр 'Adventure', жанр 'Sci-Fi', старым релизом

Road Warrior, The (Mad Max 2)
Схожесть: 0.917
Объяснение: Фильм характеризуется: жанр 'Action', жанр 'Adventure', жанр 'Sci-Fi', жанр 'Thriller'

Star Wars: Episode VI - Return of the Jedi
Схожесть: 0.917
Объяснение: Фильм характеризуется: высоким средним рейтингом, жанр 'Action', жанр 'Adventure', жанр 'Sci-Fi'

Star Trek: The Motion Picture
Схожесть: 0.912
Объяснение: Фильм характеризуется: жанр 'Adventure', жанр 'Sci-Fi', старым релизом
```

Рисунок 4.7.2 – Интерпретация полученных рекомендаций

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной практической работы успешно достигнута поставленная цель: разработана и реализована рекомендательная система, с акцентом на метод главных компонент для выявления скрытых семантических паттернов в данных.

Разработана функциональная консольная система на языке Python, интегрирующая интерактивный кинотеатр на основе датасета MovieLens и несколько стратегий рекомендаций, включая коллаборативную фильтрацию, SVD и ключевой метод — PCA. Система обеспечивает автоматическое построение PCA-модели на основе комплексной матрицы признаков фильмов, включающей жанровые, временные и рейтинговые характеристики, а также визуализацию объясненной дисперсии и кластеризацию в пространстве главных компонент.

Метод главных компонент показал высокую эффективность для семантического анализа характеристик фильмов и снижения размерности признакового пространства. Автоматически построенные компоненты обладают хорошей интерпретируемостью, что обеспечивает прозрачность и объяснимость рекомендаций.

СПИСОК ИНФОРМАЦИОННЫХ ИСТОЧНИКОВ

1. Сорокин, А. Б. Безусловная оптимизация. [Электронный ресурс] : учебно-метод. пособие / А. Б. Сорокин, О. В. Платонова, Л. М. Железняк — М. РТУ МИРЭА , 2020.
2. Рекомендательные системы: user-based и item-based [Электронный ресурс]: Habr. URL: <https://habr.com/ru/companies/surfingbird/articles/139518/> (Дата обращения: 24.10.2025).
3. Рекомендательные системы [Электронный ресурс]: URL: https://neerc.ifmo.ru/wiki/index.php?title=Рекомендательные_системы (Дата обращения: 22.10.2025).
4. MovieLens [Электронный ресурс]: URL: <https://grouplens.org/datasets/movielens/> (Дата обращения: 13.10.2025).
5. Сингулярное разложение — это очень полезно! [Электронный ресурс]: URL: <https://kpfu.ru/portal/docs/F984109526/SVD.pdf> (Дата обращения: 29.10.2025).
6. Сингулярное разложение матрицы [Электронный ресурс]: URL: <https://vmath.ru/vf5/algebra2/svd> (Дата обращения: 12.11.2025).
7. Метод главных компонент (PCA). Принцип работы и реализация с нуля на Python [Электронный ресурс]: URL: <https://habr.com/ru/articles/799001/> (Дата обращения: 16.11.2025).
8. Метод главных компонент [Электронный ресурс]: URL: [https://neerc.ifmo.ru/wiki/index.php?title=%D0%9C%D0%B5%D1%82%D0%BE%D0%B4_%D0%B3%D0%BB%D0%B0%D0%B2%D0%BD%D1%8B%D1%85_%D0%BA%D0%BE%D0%BC%D0%BF%D0%BE%D0%BD%D0%B5%D0%BD%D1%82_\(PCA\)](https://neerc.ifmo.ru/wiki/index.php?title=%D0%9C%D0%B5%D1%82%D0%BE%D0%B4_%D0%B3%D0%BB%D0%B0%D0%B2%D0%BD%D1%8B%D1%85_%D0%BA%D0%BE%D0%BC%D0%BF%D0%BE%D0%BD%D0%B5%D0%BD%D1%82_(PCA)) (Дата обращения: 15.11.2025).

ПРИЛОЖЕНИЯ

Приложение А — Реализация кинотеатра с встроенной рекомендательной системой.

Приложение Б — Код файла MarkovChain.py.

Приложение В — Код файла ContinuousMarkovChain.py.

Приложение Г — Код файла SVDRecommender.py.

Приложение Д — Код файла PCARRecommender.py.

Приложение А

Реализация кинотеатра с встроенной рекомендательной системой.

Листинг А – Реализация кинотеатра с встроенной рекомендательной системой

```
import os
import re
import pandas as pd
import numpy as np
from rich.console import Console
from rich.table import Table
from rich.prompt import Confirm, Prompt
from rich.panel import Panel
from rich.align import Align
from rich.text import Text
from enum import Enum
import seaborn as sns
import matplotlib.pyplot as plt
from datetime import datetime, timedelta

from MarkovChain import MarkovChainRecommender
from ContinuousMarkovChain import ContinuousMarkovRecommender
from SVDRecommender import SVDRecommender
from PCARRecommender import PCARRecommender

def extract_year(title):
    title = title.strip()
    match = re.search(r"\\((\\d{4})\\)\\s*$", title)
    if match:
        return int(match.group(1))
    return np.nan

class Genres(Enum):
    Action = "Action"
    Adventure = "Adventure"
    Animation = "Animation"
    Children = "Children's"
    Comedy = "Comedy"
    Crime = "Crime"
    Documentary = "Documentary"
    Drama = "Drama"
    Fantasy = "Fantasy"
    FilmNoir = "Film-Noir"
    Horror = "Horror"
    Musical = "Musical"
    Mystery = "Mystery"
    Romance = "Romance"
    SciFi = "Sci-Fi"
    Thriller = "Thriller"
    War = "War"
    Western = "Western"

class MovieLensCinema:
    def __init__(self, path, per_page=10, enable_markov_visualization=True):
        self.path = path
        self.links = None
        self.movies = None
        self.ratings = None
```

```

self.tags = None
self.console = Console()
self.rates = pd.DataFrame(columns=["movieId",
"rating"]).set_index("movieId")
self.load_data()

self.per_page = per_page
self.markov_recommender = MarkovChainRecommender(
    self, auto_visualize=enable_markov_visualization
)
self.continuous_markov_recommender = ContinuousMarkovRecommender(
    self, auto_visualize=enable_markov_visualization
)
self.svd = SVDRecommender(self, k=40)
self.pca_recommender = PCARecommender(self)

def load_data(self):
    links_path = os.path.join(self.path, "links.csv")
    movies_path = os.path.join(self.path, "movies.csv")
    ratings_path = os.path.join(self.path, "ratings.csv")
    tags_path = os.path.join(self.path, "tags.csv")

    if os.path.exists(links_path):
        self.links = pd.read_csv(
            links_path,
            encoding="utf-8",
            index_col="movieId",
            dtype={"imdbId": "int64", "tmdbId": "Int64"},
        )
    else:
        raise FileNotFoundError("Не найдено файла links.csv")

    if os.path.exists(movies_path):
        self.movies = pd.read_csv(
            movies_path, encoding="utf-8", index_col="movieId",
quotechar='"'
        )
        self.movies["title"] = self.movies["title"].str.strip()
        self.movies["genres"] = self.movies["genres"].apply(
            lambda x: [] if x == "(no genres listed)" else x.split("|")
        )
        self.movies["year"] = self.movies["title"].apply(extract_year)
        self.movies["year"] = self.movies["year"].astype("Int64")
        self.movies["title"] = self.movies["title"].apply(
            lambda t: re.sub(r"\s*(\d{4})$", "", t)
        )
    else:
        raise FileNotFoundError("Не найдено файла movies.csv")

    if os.path.exists(ratings_path):
        self.ratings = pd.read_csv(
            ratings_path,
            encoding="utf-8",
            dtype={
                "userId": "int32",
                "movieId": "int32",
                "rating": "float32",
                "timestamp": "int64",
            },
        )
        self.ratings["datetime"] = pd.to_datetime(

```

```

        self.ratings["timestamp"], unit="s"
    )
    self.ratings = self.ratings.drop(columns=["timestamp"])
    self.ratings.set_index(["userId", "movieId"], inplace=True)

else:
    raise FileNotFoundError("Не найдено файла ratings.csv")

if os.path.exists(tags_path):
    self.tags = pd.read_csv(
        tags_path,
        dtype={
            "userId": "int32",
            "movieId": "int32",
            "tag": "string",
            "timestamp": "int64",
        },
        quotechar='"',
    )
    self.tags["datetime"] = pd.to_datetime(self.tags["timestamp"],
unit="s")
    self.tags.set_index(["userId", "movieId"], inplace=True)
else:
    raise FileNotFoundError("Не найдено файла tags.csv")

def get_rating_matrix(self):
    df = self.ratings.reset_index()

    rating_matrix = df.pivot_table(
        index="userId", columns="movieId", values="rating"
    )

    return rating_matrix

def show_movies(self, page=1):
    start = (page - 1) * self.per_page
    end = start + self.per_page
    subset = self.movies.iloc[start:end]

    table = Table(title=f"🎬 Список фильмов — страница {page}")

    table.add_column("ID", style="cyan", no_wrap=True)
    table.add_column("Название", style="bold")
    table.add_column("Год", justify="center", style="magenta")
    table.add_column("Жанры", style="green")

    for movie_id, row in subset.iterrows():
        genres_str = ", ".join(row["genres"]) if row["genres"] else "-"
        year_str = str(row["year"]) if not pd.isna(row["year"]) else "-"
        table.add_row(str(movie_id), row["title"], year_str, genres_str)

    self.console.print(table)

def run(self):
    page = 1
    total_pages = (len(self.movies) - 1) // self.per_page + 1

    while True:
        os.system("cls" if os.name == "nt" else "clear")
        self.show_movies(page=page)

```

Продолжение Листинга А

```
self.show_paginator(page, total_pages)

self.console.print(
    "[yellow]n[/yellow] – следующая, [yellow]p[/yellow] –
предыдущая, "
    "[yellow]число[/yellow] – перейти на страницу\n"
    "[yellow]f[/yellow] – поиск\n"
    "[yellow]s[/yellow] – отсортировать результаты\n"
    "[yellow]o[/yellow] – перейти на страницу фильма\n"
    "[yellow]b[/yellow] – вернуться на главную страницу\n"
    "[yellow]q[/yellow] – выход"
)

choice = input("> ").strip().lower()
if choice == "n" and page < total_pages:
    page += 1
elif choice == "p" and page > 1:
    page -= 1
elif choice.isdigit():
    num = int(choice)
    if 1 <= num <= total_pages:
        page = num
elif choice == "f":
    self.search_movies()
elif choice == "s":
    self.sort_movies(self.movies)
elif choice == "o":
    movie_id_str = Prompt.ask("Введите ID фильма (или 'b' для
отмены)")
    if movie_id_str.lower() == "b":
        continue
    if movie_id_str.isdigit():
        self.show_movie_page(int(movie_id_str))
    else:
        self.console.print("[red]Некорректный ID[/red]")
        input("Нажмите Enter, чтобы вернуться...")
elif choice == "b":
    break
elif choice == "q":
    if Confirm.ask("Are you sure?"):
        exit(0)

def menu(self):
    while True:
        os.system("cls" if os.name == "nt" else "clear")

        self.console.print(
            Panel.fit(
                "[bold magenta]🎬 Добро пожаловать в консольный
кинотеатр![/bold magenta]\nВыберите действие:"
            ),
            justify="center",
        )

        self.console.print("[cyan]1.[/cyan] 🍿 Подборка фильмов")
        self.console.print("[cyan]2.[/cyan] 🔍 Мои рекомендации")
        self.console.print("[cyan]3.[/cyan] 🗣️ Мои оценки")
        self.console.print(
            "[cyan]4.[/cyan] 📊 График зависимости качества рекомендаций от
числа скрытых факторов (SVD)"
        )
```

```

    )
    self.console.print("[cyan]5.[/cyan] 🧠 Кластеризация пользователей
(SVD) ")
    self.console.print("[cyan]6.[/cyan] 📊 Анализ главных компонент
(PCA) ")
    self.console.print("[cyan]7.[/cyan] ❌ Выход")

    choice = Prompt.ask(
        "\nВыберите пункт", choices=["1", "2", "3", "4", "5", "6", "7"]
    )

    if choice == "1":
        self.run()
    elif choice == "2":
        self.show_recommendations()
    elif choice == "3":
        self.show_my_ratings()
    elif choice == "4":
        self.svd_analyze()
    elif choice == "5":
        self.cluster_menu()
    elif choice == "6":
        self.pca_analysis_menu()
    elif choice == "7":
        if Confirm.ask("[bold red]Вы действительно хотите выйти?[/bold
red]"):
            break

def show_movie_page(self, movie_id: int):
    """Страница фильма с информацией, тегами и возможностью оценки"""
    if movie_id not in self.movies.index:
        self.console.print(f"[red]Фильм с ID {movie_id} не найден.[/red]")
        input("Нажмите Enter, чтобы вернуться...")
        return

    while True:
        os.system("cls" if os.name == "nt" else "clear")

        movie = self.movies.loc[movie_id]

        info_table = Table(show_header=False, box=None)
        info_table.add_row(
            "📋 Название:", f"[bold cyan]{movie['title']}[/bold cyan]"
        )
        info_table.add_row(
            "📅 Год:", str(movie["year"]) if movie["year"] == movie["year"]
else "-"
        )
        info_table.add_row("🎭 Жанры:", f"[green]{movie['genres']}[/green]")
        self.console.print(Panel(info_table, title=f"ID {movie_id}",
expand=False))

        if self.tags is not None:
            try:
                df = self.tags.xs(movie_id, level="movieId")
                movie_tags = df["tag"].tolist()
            except KeyError:
                movie_tags = []

            if movie_tags:
```

Продолжение Листинга А

```
tags_str = ", ".join(f"[yellow]{t}[/yellow]" for t in
movie_tags)
self.console.print(Panel(tags_str, title="Теги",
expand=False))
else:
    self.console.print(
        Panel(
            "[italic grey]Для этого фильма нет тегов[/italic
grey]",
            title="Теги",
            expand=False,
        )
    )

self.console.print(
    "\n[bold]Доступные действия:[/bold]\n"
    "[green]r[/green] – оценить фильм\n"
    "[yellow]b[/yellow] – назад\n"
    "[red]q[/red] – выйти из приложения"
)

choice = input("> ").strip().lower()
if choice == "r":
    self.rate_movie(movie_id)
elif choice == "b":
    break
elif choice == "q":
    if Confirm.ask("Are you sure?"):
        exit(0)

def rate_movie(self, movie_id: int):
    """Простейшая система выставления оценки"""
    while True:
        rating = Prompt.ask(
            "Введите оценку от 0.5 до 5.0 (или 'b' чтобы вернуться)"
        )
        if rating.lower() == "b":
            break
        try:
            rating = float(rating)
            if 0.5 <= rating <= 5.0:
                self.rates.loc[movie_id, "rating"] = rating
                self.markov_recommender.reset_matrix()
                self.continuous_markov_recommender.reset_matrix()
                self.markov_recommender.build_transition_matrix()
                self.continuous_markov_recommender.build_intensity_matrix()
                self.console.print(
                    f"[bold green]Спасибо! Ваша оценка {rating} сохранена
для фильма с ID {movie_id}[/bold green]"
                )
                input("Нажмите Enter, чтобы вернуться...")
                break
            else:
                self.console.print(
                    "[red]Оценка должна быть в диапазоне 0.5–5.0[/red]"
                )
                input("Нажмите Enter, чтобы вернуться...")
                break
        except ValueError:
            self.console.print("[red]Введите корректное число или
'b'[/red]")
```

Продолжение Листинга А

```
        input("Нажмите Enter, чтобы вернуться...")
        break

    def search_movies(self):
        """Поиск фильмов по названию с фильтрацией по году и жанрам"""
        os.system("cls" if os.name == "nt" else "clear")
        search_query = (
            Prompt.ask("Введите название фильма (Enter –
пропустить)").strip().lower()
        )
        selected_genres = self.choose_genres()
        year_op, year_val = self.choose_year_filter()

    def apply_filters():
        df = self.movies
        if search_query:
            df = df[df["title"].str.lower().str.contains(search_query,
na=False)]
        if selected_genres:
            df = df[
                df["genres"].apply(
                    lambda g: all(gen in g for gen in selected_genres)
                )
            ]
        if year_op is not None and year_val is not None:
            if year_op == "=":
                df = df[df["year"] == year_val]
            elif year_op == ">":
                df = df[df["year"] > year_val]
            elif year_op == "<":
                df = df[df["year"] < year_val]
        return df

    filtered_df = apply_filters()
    if filtered_df.empty:
        self.console.print(
            Panel("[red]❌ Фильмы не найдены[/red]", title="Результат")
        )
        input("Нажмите Enter, чтобы вернуться...")
        return

    page = 1
    per_page = self.per_page

    while True:
        os.system("cls" if os.name == "nt" else "clear")

        total_pages = max(1, (len(filtered_df) - 1) // per_page + 1)
        page = min(page, total_pages)

        start = (page - 1) * per_page
        end = start + per_page
        page_data = filtered_df.iloc[start:end]

        table = Table(
            title=f"Результаты поиска: {len(filtered_df)}",
            show_header=True,
            header_style="bold magenta",
        )
        table.add_column("ID", style="cyan", width=6)
        table.add_column("Название", style="white")
```

Продолжение Листинга А

```
table.add_column("Год", style="yellow", width=8)
table.add_column("Жанры", style="green")

for idx, row in page_data.iterrows():
    year = str(row["year"]) if not pd.isna(row["year"]) else "-"
    genres = ", ".join(row["genres"]) if row["genres"] else "-"
    table.add_row(str(idx), row["title"], year, genres)

self.console.print(table)

self.show_paginator(page, total_pages)

status = f"[bold]Фильтры:[/bold] Название: [cyan]{search_query or '-'}
'[/cyan], Жанры: [cyan]{'', '.join(selected_genres) if selected_genres else '-'}
'[/cyan], Год: [cyan]{year_op + str(year_val) if year_op and year_val else '-'}
'[/cyan]"
self.console.print(status)

self.console.print(
    "[yellow]n[/yellow] - следующая, [yellow]p[/yellow] -
предыдущая, [yellow]число[/yellow] - перейти на страницу\n"
    "[yellow]s[/yellow] - отсортировать результаты\n"
    "[yellow]o[/yellow] - открыть страницу фильма по ID\n"
    "[yellow]r[/yellow] - сброс фильтров\n"
    "[yellow]b[/yellow] - назад"
)

choice = input("> ").strip().lower()
if choice == "n" and page < total_pages:
    page += 1
elif choice == "p" and page > 1:
    page -= 1
elif choice == "s":
    self.sort_movies(filtered_df)
elif choice.isdigit():
    num = int(choice)
    if 1 <= num <= total_pages:
        page = num
elif choice == "o":
    movie_id_str = Prompt.ask("Введите ID фильма (или 'b' чтобы
вернуться)")
    if movie_id_str.lower() == "b":
        continue
    try:
        movie_id = int(movie_id_str)
        if movie_id in self.movies.index:
            self.show_movie_page(movie_id)
        else:
            self.console.print("[red]Фильм с таким ID не
найден[/red]")
            input("Нажмите Enter, чтобы продолжить...")
    except ValueError:
        self.console.print("[red]Введите корректный ID[/red]")
        input("Нажмите Enter, чтобы продолжить...")
elif choice == "r":
    search_query = ""
    selected_genres = []
    year_op, year_val = None, None
    filtered_df = self.movies
    page = 1
elif choice == "b":
```

```

        break

    def choose_genres(self):
        """Выбор одного или нескольких жанров через консоль с таблицей"""
        genre_list = list(Genres)
        table = Table(
            title="Выберите жанры", show_header=True, header_style="bold
magenta"
        )
        table.add_column("№", justify="center", style="cyan", width=4)
        table.add_column("Жанр", justify="left", style="green")

        for i, genre in enumerate(genre_list, 1):
            table.add_row(str(i), genre.value)

        self.console.print(table)
        choice = Prompt.ask(
            "Введите номера жанров через пробел (Enter — пропустить)"
        ).strip()
        if not choice:
            return []

        selected_genres = []
        for num in choice.split():
            num = num.strip()
            if num.isdigit():
                idx = int(num) - 1
                if 0 <= idx < len(genre_list):
                    selected_genres.append(genre_list[idx].value)
        return selected_genres

    def choose_year_filter(self):
        """Выбор фильтра по году с оператором"""
        op = Prompt.ask(
            "Выберите оператор для фильтра по году",
            choices=[">", "<", "="],
            default="=",
        )
        year_str = Prompt.ask("Введите год").strip()
        if not year_str.isdigit():
            self.console.print("[red]Некорректный год, фильтр не будет
применён[/red]")
            return None, None
        return op, int(year_str)

    def show_paginator(self, page, total_pages):
        """Красивый центрированный пагинатор"""
        paginator_text = Text()
        last_was_ellipsis = False

        if page > 1:
            paginator_text.append("◀ Prev ", style="bold yellow")
        else:
            paginator_text.append(" ")

        for p in range(1, total_pages + 1):
            if p == 1 or p == total_pages or abs(p - page) <= 1:
                if p == page:
                    paginator_text.append(f" {p} ", style="reverse green")
                else:
                    paginator_text.append(f" {p} ", style="bold cyan")

```

```

        last_was_ellipsis = False
    else:
        if not last_was_ellipsis:
            paginator_text.append(" ... ", style="dim")
            last_was_ellipsis = True

    if page < total_pages:
        paginator_text.append(" Next ►", style="bold yellow")

    self.console.print(Align.center(paginator_text))

    def show_my_ratings(self):
        if self.rates.empty:
            self.console.print(
                Panel(
                    "[red]Вы еще не оценили ни одного фильма[/red]", title="Ваши
оценки"
                )
            )
            input("Нажмите Enter, чтобы вернуться...")
            return

        page = 1
        df = self.rates.merge(self.movies, on="movieId")
        total_pages = (len(df) - 1) // self.per_page + 1

        while len(df):
            os.system("cls" if os.name == "nt" else "clear")

            start = (page - 1) * self.per_page
            end = start + self.per_page
            subset = df.iloc[start:end]

            rates = Table(show_header=True, title=f"Ваши оценки – страница
{page}")

            rates.add_column("ID", style="cyan", no_wrap=True)
            rates.add_column("Название", style="bold")
            rates.add_column("Год", justify="center", style="magenta")
            rates.add_column("Жанры", style="green")
            rates.add_column("Оценка", style="green")

            for movie_id, row in subset.iterrows():
                genres_str = ", ".join(row["genres"]) if row["genres"] else "-"
                year_str = str(row["year"]) if not pd.isna(row["year"]) else "-"
                rates.add_row(
                    str(movie_id),
                    row["title"],
                    year_str,
                    genres_str,
                    str(row["rating"]),
                )

            self.console.print(rates)

            self.show_paginator(page, total_pages)

            self.console.print(
                "[yellow]n[/yellow] – следующая, [yellow]p[/yellow] –
предыдущая, "
                "[yellow]число[/yellow] – перейти на страницу\n"
                "[yellow]d[/yellow] – удалить оценку\n"

```

```

        "[yellow]o[/yellow] - перейти на страницу фильма\n"
        "[yellow]b[/yellow] - вернуться на главную страницу\n"
        "[yellow]q[/yellow] - выход"
    )

    choice = input("> ").strip().lower()
    if choice == "n" and page < total_pages:
        page += 1
    elif choice == "p" and page > 1:
        page -= 1
    elif choice.isdigit():
        num = int(choice)
        if 1 <= num <= total_pages:
            page = num
    elif choice == "d":
        movie_id_str = Prompt.ask(
            "Введите ID фильма, у которого хотите удалить оценку (или
'b' для отмены)"
        )
        if movie_id_str.lower() == "b":
            continue
        if movie_id_str.isdigit():
            id = int(movie_id_str)
            if id in self.rates.index:
                self.rates = self.rates.drop(index=id)
                df = df.drop(index=id)
                self.markov_recommender.reset_matrix()
                self.markov_recommender.build_transition_matrix()
            else:
                self.console.print(
                    "[red]Не найдено оценки для фильма с заданным
ID[/red]"
                )
                input("Нажмите Enter, чтобы вернуться...")
        else:
            self.console.print("[red]Некорректный ID[/red]")
            input("Нажмите Enter, чтобы вернуться...")
    elif choice == "o":
        movie_id_str = Prompt.ask("Введите ID фильма (или 'b' для
отмены)")
        if movie_id_str.lower() == "b":
            continue
        if movie_id_str.isdigit():
            self.show_movie_page(int(movie_id_str))
        else:
            self.console.print("[red]Некорректный ID[/red]")
            input("Нажмите Enter, чтобы вернуться...")
    elif choice == "b":
        break
    elif choice == "q":
        if Confirm.ask("Are you sure?"):
            exit(0)

def sort_movies(self, frame):
    """Сортировка фильмов по различным полям"""
    sort_fields = [
        ("ID", "movieId"),
        ("Название", "title"),
        ("Год", "year"),
        ("Жанры", "genres"),
    ]

```

```
table = Table(
    title="Выберите поле для сортировки",
    show_header=True,
    header_style="bold magenta",
)
table.add_column("№", justify="center", style="cyan", width=4)
table.add_column("Поле", style="green")

for i, (label, _) in enumerate(sort_fields, 1):
    table.add_row(str(i), label)

self.console.print(table)

choice = Prompt.ask("Введите номер поля (или Enter для отмены)").strip()

if not choice.isdigit():
    return

choice_num = int(choice)
if not 1 <= choice_num <= len(sort_fields):
    self.console.print("[red]Некорректный выбор[/red]")
    input("Нажмите Enter, чтобы вернуться...")
    return

field_label, field_name = sort_fields[choice_num - 1]

direction = Prompt.ask(
    "Выберите направление сортировки", choices=["asc", "desc"],
default="asc"
)

ascending = direction == "asc"

if field_name == "movieId":
    sorted_df = frame.sort_index(ascending=ascending)
elif field_name == "genres":
    sorted_df = frame.copy()
    sorted_df["__sort_genre"] = sorted_df["genres"].apply(
        lambda g: g[0] if g else ""
    )
    sorted_df = sorted_df.sort_values("__sort_genre",
ascending=ascending).drop(
        columns="__sort_genre"
    )
else:
    sorted_df = frame.sort_values(field_name, ascending=ascending)

self.paginated_view(
    sorted_df, title=f"Сортировка по {field_label} ({direction})"
)

def paginated_view(self, df, title="Список фильмов"):
    """Универсальный постраничный просмотр DataFrame фильмов"""
    page = 1
    total_pages = max(1, (len(df) - 1) // self.per_page + 1)

    while True:
        os.system("cls" if os.name == "nt" else "clear")
        start = (page - 1) * self.per_page
        end = start + self.per_page
```

```

subset = df.iloc[start:end]

table = Table(title=f"{title} – страница {page}", show_header=True)
table.add_column("ID", style="cyan", no_wrap=True)
table.add_column("Название", style="bold")
table.add_column("Год", justify="center", style="magenta")
table.add_column("Жанры", style="green")

for movie_id, row in subset.iterrows():
    genres_str = ", ".join(row["genres"]) if row["genres"] else "-"
    year_str = str(row["year"]) if not pd.isna(row["year"]) else "-"
    table.add_row(str(movie_id), row["title"], year_str, genres_str)

self.console.print(table)
self.show_paginator(page, total_pages)
self.console.print(
    "[yellow]n[/yellow] – следующая, [yellow]p[/yellow] –
предыдущая, "
    "[yellow]число[/yellow] – перейти на страницу\n"
    "[yellow]o[/yellow] – открыть страницу фильма,
[yellow]b[/yellow] – назад"
)

choice = input("> ").strip().lower()
if choice == "n" and page < total_pages:
    page += 1
elif choice == "p" and page > 1:
    page -= 1
elif choice.isdigit():
    num = int(choice)
    if 1 <= num <= total_pages:
        page = num
elif choice == "o":
    movie_id_str = Prompt.ask("Введите ID фильма (или 'b' для
отмены)")
    if movie_id_str.lower() == "b":
        continue
    if movie_id_str.isdigit():
        self.show_movie_page(int(movie_id_str))
elif choice == "b":
    break
os.system("cls" if os.name == "nt" else "clear")

def show_recommendations(self):
    """Меню выбора стратегии рекомендаций"""
    os.system("cls" if os.name == "nt" else "clear")
    if self.rates.empty:
        self.console.print(
            Panel(
                "[red]У вас нет оценок для генерации рекомендаций.[/red]",
                title="Рекомендации",
            )
        )
    input("Нажмите Enter, чтобы вернуться...")
    return

strategy = Prompt.ask(
    "Выберите стратегию рекомендаций",
    choices=[
        "user-based",
        "item-based",
    ]
)

```

```

        "markov-discrete",
        "markov-continuous",
        "svd",
        "pca",
    ],
    default="user-based",
)

if strategy == "user-based":
    self.recommend_user_based()
elif strategy == "item-based":
    self.recommend_item_based()
elif strategy == "markov-discrete":
    self.recommend_markov_chain()
elif strategy == "svd":
    self.recommend_svd()
elif strategy == "pca":
    self.recommend_pca()
else:
    self.recommend_continuous_markov_chain()

def recommend_user_based(self):
    """User-based рекомендации"""
    R = self.get_rating_matrix()
    user_ids = R.index.tolist()
    movie_ids = R.columns.tolist()
    current_user_id = -1

    current_user_ratings = pd.Series(
        [np.nan] * len(movie_ids), index=movie_ids, dtype=float
    )
    for movie_id in self.rates.index:
        if movie_id in movie_ids:
            current_user_ratings[movie_id] = self.rates.loc[movie_id,
"rating"]

    R = pd.concat(
        [R, pd.DataFrame([current_user_ratings], index=[current_user_id])]
    )
    current_user_idx = R.index.get_loc(current_user_id)

    metric = Prompt.ask(
        "Выберите метрику сходства [pearson/jaccard/lp/otiai]",
        choices=["pearson", "jaccard", "lp", "otiai"],
        default="pearson",
    )

    R_values = R.values.astype(np.float64)
    n_users, n_items = R_values.shape
    user_means = np.nanmean(R_values, axis=1)
    preds = np.full(n_items, np.nan)
    sims = np.zeros(n_users)
    current_ratings = R_values[current_user_idx, :]

    for u in range(n_users):
        if u == current_user_idx:
            continue
        other = R_values[u, :]
        mask = ~np.isnan(current_ratings) & ~np.isnan(other)
        if np.sum(mask) == 0:
            continue

```

```

        if metric == "pearson":
            sims[u] = np.corrcoef(current_ratings[mask], other[mask])[0, 1]
        elif metric == "lp":
            sims[u] = -np.linalg.norm(current_ratings[mask] - other[mask])
        elif metric == "jaccard":
            sims[u] = np.sum(
                (current_ratings[mask] > 0) & (other[mask] > 0)
            ) / np.sum((current_ratings[mask] > 0) | (other[mask] > 0))
        elif metric == "otiai":
            sims[u] = np.dot(current_ratings[mask], other[mask]) / (
                np.linalg.norm(current_ratings[mask]) *
np.linalg.norm(other[mask])
            )

    for i in range(n_items):
        if not np.isnan(R_values[current_user_idx, i]):
            continue
        mask = ~np.isnan(R_values[:, i])
        if np.sum(mask) == 0:
            continue
        numerator = np.sum(sims[mask] * (R_values[mask, i] -
user_means[mask]))
        denominator = np.sum(np.abs(sims[mask])) + 1e-8
        preds[i] = user_means[current_user_idx] + numerator / denominator

    recs_df = pd.DataFrame({"movieId": R.columns, "pred_rating": preds})
    recs_df = recs_df.dropna().sort_values("pred_rating", ascending=False)
    self.paginated_view(
        self.movies.loc[recs_df["movieId"]], title=f"User-based ({metric})"
    )

def recommend_item_based(self):
    """Item-based рекомендации"""
    R = self.get_rating_matrix()
    user_ids = R.index.tolist()
    movie_ids = R.columns.tolist()
    current_user_id = -1

    current_user_ratings = pd.Series(
        [np.nan] * len(movie_ids), index=movie_ids, dtype=float
    )
    for movie_id in self.rates.index:
        if movie_id in movie_ids:
            current_user_ratings[movie_id] = self.rates.loc[movie_id,
"rating"]

    R = pd.concat(
        [R, pd.DataFrame([current_user_ratings], index=[current_user_id])]
    )
    current_user_idx = R.index.get_loc(current_user_id)

    metric = Prompt.ask(
        "Выберите метрику сходства [pearson/jaccard/lp/otiai]",
        choices=["pearson", "jaccard", "lp", "otiai"],
        default="pearson",
    )

    R_values = R.values.astype(np.float64)
    n_items = R_values.shape[1]
    item_means = np.nanmean(R_values, axis=0)
    preds = np.full(n_items, np.nan)

```

```

        for i in range(n_items):
            if not np.isnan(R_values[current_user_idx, i]):
                continue
            rated_mask = ~np.isnan(R_values[current_user_idx, :])
            sims_i = []
            ratings_i = []
            for j in np.where(rated_mask)[0]:
                mask = ~np.isnan(R_values[:, i]) & ~np.isnan(R_values[:, j])
                if np.sum(mask) == 0:
                    sim = 0
                else:
                    if metric == "pearson":
                        sim = np.corrcoef(R_values[mask, i], R_values[mask,
j]))[0, 1]

                    elif metric == "lp":
                        sim = -np.linalg.norm(R_values[mask, i] - R_values[mask,
j])

                    elif metric == "jaccard":
                        sim = np.sum(
                            (R_values[mask, i] > 0) & (R_values[mask, j] > 0)
                        ) / np.sum((R_values[mask, i] > 0) | (R_values[mask, j]
> 0))

                    elif metric == "otiai":
                        sim = np.dot(R_values[mask, i], R_values[mask, j]) / (
                            np.linalg.norm(R_values[mask, i])
                            * np.linalg.norm(R_values[mask, j])
                        )
                sims_i.append(sim)
                ratings_i.append(R_values[current_user_idx, j] - item_means[j])
            sims_i = np.array(sims_i)
            ratings_i = np.array(ratings_i)
            if np.sum(np.abs(sims_i)) > 0:
                preds[i] = item_means[i] + np.dot(sims_i, ratings_i) / np.sum(
                    np.abs(sims_i)
                )

        recs_df = pd.DataFrame({"movieId": R.columns, "pred_rating": preds})
        recs_df = recs_df.dropna().sort_values("pred_rating", ascending=False)
        self.paginated_view(
            self.movies.loc[recs_df["movieId"]], title=f"Item-based ({metric})"
        )

    def recommend_markov_chain(self):
        """Улучшенные рекомендации с помощью дискретной цепи Маркова"""
        if self.rates.empty:
            self.console.print(
                Panel("[red]Вы не оценили ни одного фильма![/red]",
title="Ошибка")
            )
            input("Нажмите Enter, чтобы вернуться...")
            return

        if len(self.rates) < 2:
            self.console.print(
                Panel(
                    "[yellow]Оцените хотя бы 2 фильма для построения цепи
Маркова[/yellow]",
                    title="Недостаточно данных",
                )
            )

```

```
        unrated_movies =
self.movies[~self.movies.index.isin(self.rates.index)]
        if not unrated_movies.empty:
            if Confirm.ask("Показать случайные рекомендации вместо цепи
Маркова?"):
                recommendations = unrated_movies.sample(
                    min(20, len(unrated_movies))
                )
                self.paginated_view(recommendations, title="Случайные
рекомендации")
            else:
                self.console.print("[red]Нет фильмов для рекомендаций[/red]")
                input("Нажмите Enter, чтобы вернуться...")
                return

        markov_rec = self.markov_recommender

        steps = Prompt.ask(
            "Количество шагов цепи Маркова", choices=["1", "2", "3"],
default="1"
        )
        steps = int(steps)

        top_k = Prompt.ask("Количество рекомендаций", default="20")
        top_k = int(top_k)

        recommendations = markov_rec.recommend_movies(steps=steps, top_k=top_k)

        if recommendations.empty:
            self.console.print(
                Panel(
                    "[yellow]Не найдено подходящих рекомендаций. Попробуйте
оценить больше фильмов разных жанров.[/yellow]",
                    title="Рекомендации",
                )
            )

            unrated_movies =
self.movies[~self.movies.index.isin(self.rates.index)]
            if not unrated_movies.empty:
                if Confirm.ask("Показать случайные рекомендации?"):
                    random_recs = unrated_movies.sample(min(20,
len(unrated_movies)))
                    self.paginated_view(random_recs, title="Случайные
рекомендации")
                else:
                    input("Нажмите Enter, чтобы вернуться...")
                    return

        if Confirm.ask("Показать матрицу переходов?"):
            os.system("cls" if os.name == "nt" else "clear")
            markov_rec.show_transition_matrix()
            input("Нажмите Enter, чтобы продолжить...")

        if Confirm.ask("Показать вероятности жанров?"):
            os.system("cls" if os.name == "nt" else "clear")
            self.show_markov_genre_probabilities(steps)
            input("Нажмите Enter, чтобы продолжить...")

        self.console.print(
```

Продолжение Листинга А

```
        Panel(
            f"
```

```
        ratings_list.append((movie_id, row["rating"], timestamp))

    ratings_list.sort(key=lambda x: x[2])
    return ratings_list

    def recommend_continuous_markov_chain(self):
        """Рекомендации на основе непрерывной цепи Маркова с временными
метками"""
        if self.rates.empty:
            self.console.print(
                Panel("[red]Вы не оценили ни одного фильма![/red]",
title="Ошибка")
            )
            input("Нажмите Enter, чтобы вернуться...")
            return

        if len(self.rates) < 2:
            self.console.print(
                Panel(
                    "[yellow]Оцените хотя бы 2 фильма для построения непрерывной
цепи Маркова[/yellow]",
                    title="Недостаточно данных",
                )
            )
            return

        continuous_markov = self.continuous_markov_recommender

        time_t = Prompt.ask("Введите время для прогноза (в днях)",
default="1.0")
        try:
            time_t = float(time_t)
        except ValueError:
            time_t = 1.0

        top_k = Prompt.ask("Количество рекомендаций", default="20")
        top_k = int(top_k)

        ratings_with_time =
self.continuous_markov_recommender.get_ratings_chronology()
        if ratings_with_time:
            self.console.print("\n[bold]Хронология ваших оценок:[/bold]")
            for i, (movie_id, rating, timestamp) in
enumerate(ratings_with_time):
                movie_title = self.movies.loc[movie_id, "title"]
                time_str = (
                    timestamp.strftime("%Y-%m-%d %H:%M")
                    if isinstance(timestamp, datetime)
                    else "недавно"
                )
                self.console.print(
                    f" {i+1}. {movie_title} - оценка {rating} - {time_str}"
                )

        recommendations = continuous_markov.recommend_movies_continuous(
            time_t=time_t, top_k=top_k
        )

        if recommendations.empty:
            self.console.print(
```

```

        Panel(
            "[yellow]Не найдено подходящих рекомендаций[/yellow]",
            title="Рекомендации",
        )
    )
    input("Нажмите Enter, чтобы вернуться...")
    return

if Confirm.ask("Показать матрицу интенсивностей?"):
    os.system("cls" if os.name == "nt" else "clear")
    continuous_markov.show_intensity_matrix()
    input("Нажмите Enter, чтобы продолжить...")

if Confirm.ask("Показать вероятности жанров для непрерывной цепи?"):
    os.system("cls" if os.name == "nt" else "clear")
    self.show_continuous_markov_genre_probabilities(time_t)
    input("Нажмите Enter, чтобы продолжить...")

self.console.print(
    Panel(
        f"🕒 Рекомендации на основе непрерывной цепи Маркова (t={time_t}
дней)",
        title="Непрерывная цепь Маркова",
    )
)

self.paginated_view(
    recommendations,
    title=f"Рекомендации (Непрерывная цепь Маркова, t={time_t} дней)",
)

def show_continuous_markov_genre_probabilities(self, time_t=1.0):
    """Показать вероятности жанров для непрерывной цепи Маркова"""
    genre_probs = (

self.continuous_markov_recommender.get_genre_probabilities_continuous(
    time_t
    )
    )

    if not genre_probs:
        self.console.print(
            "[red]Не удалось вычислить вероятности жанров для непрерывной
цепи.[/red]"
        )
        return

    sorted_genres = sorted(genre_probs.items(), key=lambda x: x[1],
reverse=True)

    prob_table = Table(title=f"Вероятности жанров (непрерывная цепь,
t={time_t})")
    prob_table.add_column("Жанр", style="cyan")
    prob_table.add_column("Вероятность", style="green")
    prob_table.add_column("Процент", style="magenta")

    for genre, prob in sorted_genres:
        if prob > 0.001:
            percentage = prob * 100
            prob_table.add_row(genre, f"{prob:.3f}", f"{percentage:.1f}%")

```

Продолжение Листинга А

```
self.console.print(prob_table)

rated_movies = self.movies.loc[self.rates.index]
if not rated_movies.empty:
    last_movie = rated_movies.iloc[-1]
    self.console.print(
        f"\n[bold]Последний оцененный фильм:[/bold]
[cy]{last_movie['title']}[/cy]"
    )
    self.console.print(
        f"[bold]Жанры:[/bold] [green]{'',
'.join(last_movie['genres'])}[/green]"
    )

def recommend_svd(self):
    os.system("cls" if os.name == "nt" else "clear")
    self.console.print(
        f"[bold cyan]🔴 SVD-модель ({self.svd.k} скрытых факторов)
формируется...[/bold cyan]"
    )

    self.svd.build()
    mse, rmse, mae = self.svd.evaluate()

    self.console.print(f"\n[green]MSE:[/green] {mse:.4f}")
    self.console.print(f"[green]RMSE:[/green] {rmse:.4f}")
    self.console.print(f"[green]MAE:[/green] {mae:.4f}\n")

    top_k = Prompt.ask("Количество рекомендаций", default="20")
    top_k = int(top_k)

    recs = self.svd.recommend_for_current_user(top_k=top_k)

    self.paginated_view(recs, title="SVD рекомендации")

def plot_similarity_metrics(
    self, strategy: str = "user-based", test_ratio: float = 0.2
):
    metrics = ["pearson", "lp", "jaccard", "otiai"]
    results = []

    for metric in metrics:
        rmse, mae = self._compute_metrics_for_plot(metric, strategy,
test_ratio)
        results.append({"metric": metric, "RMSE": rmse, "MAE": mae})

    df = pd.DataFrame(results)

    sns.set_theme(style="whitegrid", font_scale=1.1)
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))

    sns.barplot(x="metric", y="RMSE", data=df, ax=axes[0],
palette="Blues_d")
    axes[0].set_title(f"Сравнение RMSE ({strategy})")
    axes[0].set_xlabel("Метрика близости")
    axes[0].set_ylabel("RMSE")

    sns.barplot(x="metric", y="MAE", data=df, ax=axes[1],
palette="Greens_d")
    axes[1].set_title(f"Сравнение MAE ({strategy})")
    axes[1].set_xlabel("Метрика близости")
```

```

        axes[1].set_ylabel("MAE")

        plt.tight_layout()
        plt.show()

    def _compute_metrics_for_plot(self, metric: str, strategy: str, test_ratio:
float):
        R = self.get_rating_matrix().copy().values.astype(float)
        n_users, n_items = R.shape

        rng = np.random.default_rng(42)
        train = R.copy()
        test_mask = ~np.isnan(R) & (rng.random(R.shape) < test_ratio)
        test_true = np.full_like(R, np.nan)
        test_true[test_mask] = R[test_mask]
        train[test_mask] = np.nan

        user_means = np.nanmean(train, axis=1)
        item_means = np.nanmean(train, axis=0)
        preds = np.full_like(R, np.nan)

        for u in range(n_users):
            if strategy == "user-based":
                sims = np.zeros(n_users)
                current = train[u, :]
                for v in range(n_users):
                    if v == u:
                        continue
                    other = train[v, :]
                    mask = ~np.isnan(current) & ~np.isnan(other)
                    if np.sum(mask) == 0:
                        sims[v] = 0
                        continue
                    if metric == "pearson":
                        sims[v] = np.corrcoef(current[mask], other[mask])[0, 1]
                    elif metric == "lp":
                        sims[v] = -np.linalg.norm(current[mask] - other[mask])
                    elif metric == "jaccard":
                        sims[v] = np.sum(
                            (current[mask] > 0) & (other[mask] > 0)
                        ) / np.sum((current[mask] > 0) | (other[mask] > 0))
                    elif metric == "otiai":
                        sims[v] = np.sum(current[mask] * other[mask]) / (
                            np.linalg.norm(current[mask]) *
np.linalg.norm(other[mask])
                        )
                    else:
                        sims[v] = 0

                for i in range(n_items):
                    if not np.isnan(train[u, i]):
                        continue
                    mask = ~np.isnan(train[:, i])
                    if np.sum(mask) == 0:
                        continue
                    numerator = np.sum(sims[mask] * (train[mask, i] -
user_means[mask]))
                    denominator = np.sum(np.abs(sims[mask])) + 1e-8
                    preds[u, i] = user_means[u] + numerator / denominator
            else:

```

```

        for i in range(n_items):
            if not np.isnan(train[u, i]):
                continue
            rated_mask = ~np.isnan(train[u, :])
            sims_i = []
            ratings_i = []
            for j in np.where(rated_mask)[0]:
                mask = ~np.isnan(train[:, i]) & ~np.isnan(train[:, j])
                if np.sum(mask) == 0:
                    sim = 0
                else:
                    if metric == "pearson":
                        sim = np.corrcoef(train[mask, i], train[mask,
j]))[0, 1]

                    elif metric == "lp":
                        sim = -np.linalg.norm(train[mask, i] -
train[mask, j])

                    elif metric == "jaccard":
                        sim = np.sum(
                            (train[mask, i] > 0) & (train[mask, j] > 0)
                        ) / np.sum((train[mask, i] > 0) | (train[mask,
j] > 0))

                    elif metric == "otiai":
                        sim = np.sum(train[mask, i] * train[mask, j]) /
(
                            np.linalg.norm(train[mask, i])
                            * np.linalg.norm(train[mask, j])
                        )
                    else:
                        sim = 0
                sims_i.append(sim)
                ratings_i.append(train[u, j] - item_means[j])
            sims_i = np.array(sims_i)
            ratings_i = np.array(ratings_i)
            if np.sum(np.abs(sims_i)) > 0:
                preds[u, i] = item_means[i] + np.dot(
                    sims_i, ratings_i
                ) / np.sum(np.abs(sims_i))

        mask_eval = ~np.isnan(test_true) & ~np.isnan(preds)
        if np.sum(mask_eval) == 0:
            return np.nan, np.nan

        diff = preds[mask_eval] - test_true[mask_eval]
        rmse = np.sqrt(np.mean(diff**2))
        mae = np.mean(np.abs(diff))
        return rmse, mae

def svd_analyze(self):
    os.system("cls" if os.name == "nt" else "clear")
    self.console.print(
        "[bold cyan]Анализ влияния числа скрытых факторов SVD[/bold cyan]"
    )

    k_min = int(Prompt.ask("Минимальное число скрытых факторов",
default="2"))
    k_max = int(Prompt.ask("Максимальное число скрытых факторов",
default="50"))

    self.svd.evaluate_over_k(k_min=k_min, k_max=k_max)

```

Продолжение Листинга А

```
input("\nНажмите Enter, чтобы вернуться в меню...")

def cluster_menu(self):
    os.system("cls" if os.name == "nt" else "clear")
    self.console.print("[bold cyan]Кластеризация пользователей (SVD) [/bold cyan]\n")

    n_clusters = int(Prompt.ask("Число кластеров", default="4"))

    self.svd.build()
    labels = self.svd.cluster_users(n_clusters=n_clusters)

    self.console.print(f"\n[green]Готово! Найдено {n_clusters} кластеров. [/green]")
    self.console.print(f"Распределение: {np.bincount(labels)}")

    input("\nНажмите Enter, чтобы вернуться...")

def pca_analysis_menu(self):
    """Меню анализа главных компонент"""
    while True:
        os.system("cls" if os.name == "nt" else "clear")

        self.console.print(
            Panel.fit(
                "[bold magenta]📊 Анализ главных компонент (PCA) [/bold magenta]"
            ),
            justify="center",
        )

        self.console.print("[cyan]1. [/cyan] 🔑 Построить PCA модель")
        self.console.print("[cyan]2. [/cyan] 🎬 Рекомендации похожих фильмов")
        self.console.print("[cyan]3. [/cyan] 🔍 Анализ компонент")
        self.console.print("[cyan]4. [/cyan] 📈 График объясненной дисперсии")
        self.console.print("[cyan]5. [/cyan] 🏠 Кластеризация фильмов")
        self.console.print("[cyan]6. [/cyan] 📊 Рекомендации для меня (PCA)")
        self.console.print("[cyan]7. [/cyan] 🔍 Объяснение рекомендаций")
        self.console.print("[cyan]8. [/cyan] 🏠 Назад")

        choice = Prompt.ask(
            "\nВыберите пункт", choices=["1", "2", "3", "4", "5", "6", "7", "8"]
        )
        if choice == "1":
            self.build_pca_model()
        elif choice == "2":
            self.pca_similar_movies()
        elif choice == "3":
            self.analyze_pca_components()
        elif choice == "4":
            self.plot_pca_variance()
        elif choice == "5":
            self.pca_cluster_movies()
        elif choice == "6":
            self.recommend_pca()
        elif choice == "7":
            self.pca_analysis_menu()
```

```

        self.pca_explain_recommendations()
    elif choice == "8":
        break

def build_pca_model(self):
    """Построение PCA модели с улучшенным интерфейсом"""
    os.system("cls" if os.name == "nt" else "clear")
    self.console.print("[bold cyan]🔧 Построение PCA модели...[/bold cyan]")

    n_components = Prompt.ask(
        "Введите число компонент (Enter для автоматического выбора)",
        default=""
    )

    if n_components.isdigit():
        self.pca_recommender.n_components = int(n_components)
    else:
        self.pca_recommender.n_components = None

    success = self.pca_recommender.build_pca_model()

    if success:
        mse, rmse, mae = self.pca_recommender.evaluate_reconstruction()

        self.console.print(
            Panel(
                f"[green]✅ PCA модель успешно построена![/green]\n"
                f"Число компонент: {len(self.pca_recommender.components_)}\n"
                f"Объясненная дисперсия: {np.sum(self.pca_recommender.explained_variance_ratio_):.3f}\n"
                f"MSE восстановления: {mse:.4f}\n"
                f"RMSE восстановления: {rmse:.4f}\n"
                f"MAE восстановления: {mae:.4f}",
                title="Результаты PCA",
            )
        )
    else:
        self.console.print(
            Panel("[red]❌ Ошибка при построении PCA модели[/red]",
            title="Ошибка")
        )

    input("\nНажмите Enter, чтобы продолжить...")

def pca_similar_movies(self):
    """Рекомендация похожих фильмов через PCA с улучшенным интерфейсом"""
    os.system("cls" if os.name == "nt" else "clear")

    movie_id_str = Prompt.ask("Введите ID фильма для поиска похожих")

    if not movie_id_str.isdigit():
        self.console.print("[red]❌ Некорректный ID фильма[/red]")
        input("Нажмите Enter, чтобы продолжить...")
        return

    movie_id = int(movie_id_str)

    if movie_id not in self.movies.index:
```

```

        self.console.print("[red]❌ Фильм с таким ID не найден[/red]")
        input("Нажмите Enter, чтобы продолжить...")
        return

    top_k = Prompt.ask("Количество рекомендаций", default="10")
    similar_movies = self.pca_recommender.recommend_similar_movies(
        movie_id, int(top_k)
    )

    if not similar_movies:
        self.console.print("[yellow]❌ Не найдено похожих фильмов[/yellow]")
        input("Нажмите Enter, чтобы продолжить...")
        return

    target_movie = self.movies.loc[movie_id]
    table = Table(title=f"Фильмы, похожие на '{target_movie['title']}'")
    table.add_column("ID", style="cyan")
    table.add_column("Название", style="bold")
    table.add_column("Сходство", style="green")
    table.add_column("Жанры", style="magenta")

    for movie_id_sim, similarity in similar_movies:
        movie = self.movies.loc[movie_id_sim]
        genres_str = ", ".join(movie["genres"]) if movie["genres"] else "-"
        table.add_row(
            str(movie_id_sim), movie["title"], f"{similarity:.3f}",
genres_str
        )

    self.console.print(table)

    explanation =
self.pca_recommender.get_recommendation_explanation(movie_id)
    self.console.print(f"\n[bold]Характеристики исходного фильма:[/bold]")
    self.console.print(explanation)

    input("\nНажмите Enter, чтобы продолжить...")

def analyze_pca_components(self):
    """Анализ компонент PCA с обработкой ошибок"""
    os.system("cls" if os.name == "nt" else "clear")

    if self.pca_recommender.pca is None:
        self.console.print("[red]❌ Сначала постройте PCA модель[/red]")
        input("Нажмите Enter, чтобы продолжить...")
        return

    n_components = len(self.pca_recommender.components_)

    component_str = Prompt.ask(
        f"Введите номер компоненты (1-{n_components})", default="1"
    )

    if not component_str.isdigit():
        self.console.print("[red]❌ Некорректный номер компоненты[/red]")
        input("Нажмите Enter, чтобы продолжить...")
        return

    component_idx = int(component_str) - 1
    if component_idx < 0 or component_idx >= n_components:

```

```

        self.console.print(
            f"[red]✗ Номер компоненты должен быть от 1 до
{n_components}[/red]"
        )
        input("Нажмите Enter, чтобы продолжить...")
        return

    top_features =
self.pca_recommender.get_component_analysis(component_idx)

    if not top_features:
        self.console.print("[red]✗ Не удалось проанализировать
компоненту[/red]")
        input("Нажмите Enter, чтобы продолжить...")
        return

    table = Table(title=f"Анализ {component_idx + 1}-й главной компоненты")
    table.add_column("Признак", style="cyan")
    table.add_column("Вклад", style="bold green")
    table.add_column("Абсолютное значение", style="magenta")

    for feature, importance in top_features:
        abs_importance = abs(importance)
        if importance > 0:
            importance_str = f"[green]{importance:.4f}[/green]"
        else:
            importance_str = f"[red]{importance:.4f}[/red]"

        table.add_row(feature, importance_str, f"{abs_importance:.4f}")

    self.console.print(table)

    self.console.print(
        f"\n[bold]Интерпретация компоненты {component_idx + 1}:[/bold]"
    )

    positive_features = [feat for feat, imp in top_features[:5] if imp > 0]
    negative_features = [feat for feat, imp in top_features[:5] if imp < 0]

    interpretation = []

    if positive_features:
        pos_desc = []
        for feat in positive_features:
            if "genre_" in feat:
                genre = feat.replace("genre_", "")
                pos_desc.append(f"жанр '{genre}'")
            elif "rating_mean" in feat:
                pos_desc.append("высокий средний рейтинг")
            elif "is_old" in feat:
                pos_desc.append("старые фильмы")
            elif "is_recent" in feat:
                pos_desc.append("новые фильмы")
            elif "movie_age" in feat:
                pos_desc.append("возраст фильма")
            elif "rating_count" in feat:
                pos_desc.append("популярность (количество оценок)")

        if pos_desc:
            interpretation.append(f"Положительно связана с: {'
```

```

'.join(pos_desc)}")

    if negative_features:
        neg_desc = []
        for feat in negative_features:
            if "genre_" in feat:
                genre = feat.replace("genre_", "")
                neg_desc.append(f"жанр '{genre}'")
            elif "rating_mean" in feat:
                neg_desc.append("низкий средний рейтинг")
            elif "is_old" in feat:
                neg_desc.append("новые фильмы")
            elif "is_recent" in feat:
                neg_desc.append("старые фильмы")

        if neg_desc:
            interpretation.append(f"Отрицательно связана с: {'',
100 '.join(neg_desc)}")

    if interpretation:
        for line in interpretation:
            self.console.print(f"    • {line}")
    else:
        self.console.print("    • Компонента имеет сложную структуру")

    if hasattr(self.pca_recommender, "explained_variance_ratio_"):
        variance = (
            self.pca_recommender.explained_variance_ratio_[component_idx] *
100
        )
        self.console.print(f"\n[bold]Объясненная дисперсия:[/bold]
{variance:.2f}%")

        input("\nНажмите Enter, чтобы продолжить...")

    def plot_pca_variance(self):
        self.pca_recommender.plot_variance_explained()

    def recommend_pca(self):
        """Рекомендации на основе PCA"""
        os.system("cls" if os.name == "nt" else "clear")
        self.console.print(f"[bold cyan]📊 PCA-модель формируется...[/bold
cyan]")

        if self.pca_recommender.pca is None:
            success = self.pca_recommender.build_pca_model()
            if not success:
                self.console.print("[red]Ошибка построения PCA модели[/red]")
                input("Нажмите Enter, чтобы вернуться...")
                return

        mse, rmse, mae = self.pca_recommender.evaluate_reconstruction()

        self.console.print(f"\n[green]Качество восстановления:[/green]")
        self.console.print(f"[green]MSE:[/green] {mse:.4f}")
        self.console.print(f"[green]RMSE:[/green] {rmse:.4f}")
        self.console.print(f"[green]MAE:[/green] {mae:.4f}\n")

        top_k = Prompt.ask("Количество рекомендаций", default="20")
        top_k = int(top_k)

```

```

recs = self.pca_recommender.recommend_for_current_user(top_k=top_k)

if recs.empty:
    self.console.print("[yellow]Не найдено рекомендаций на основе
PCA[/yellow]")
    input("Нажмите Enter, чтобы вернуться...")
    return

table = Table(title="PCA рекомендации")
table.add_column("ID", style="cyan", no_wrap=True)
table.add_column("Название", style="bold")
table.add_column("Схожесть", style="green")
table.add_column("Жанры", style="magenta")
table.add_column("Объяснение", style="yellow")

for _, row in recs.iterrows():
    movie_id = row["movieId"]
    genres_str = ", ".join(row["genres"]) if row["genres"] else "-"

    explanation =
self.pca_recommender.get_recommendation_explanation(movie_id)

    table.add_row(
        str(movie_id),
        row["title"],
        f"{row['score']:.3f}",
        genres_str,
        explanation[:50] + "..." if len(explanation) > 50 else
explanation,
    )

    self.console.print(table)

    if Confirm.ask("\nПоказать подробную информацию о рекомендациях?"):
        for _, row in recs.iterrows():
            movie_id = row["movieId"]
            explanation =
self.pca_recommender.get_recommendation_explanation(
                movie_id
            )
            self.console.print(f"\n[bold cyan]{row['title']}[/bold cyan]")
            self.console.print(f"Схожесть: {row['score']:.3f}")
            self.console.print(f"Объяснение: {explanation}")

        input("\nНажмите Enter, чтобы вернуться...")

def pca_cluster_movies(self):
    """Классификация фильмов с помощью PCA"""
    if self.pca_recommender.pca is None:
        self.console.print("[red]Сначала постройте PCA модель[/red]")
        input("Нажмите Enter, чтобы продолжить...")
        return

    n_clusters = int(Prompt.ask("Число кластеров", default="6"))
    method = Prompt.ask(
        "Метод кластеризации",
        choices=["agglomerative", "gmm"],
        default="agglomerative",
    )

    labels = self.pca_recommender.cluster_movies(

```

```

        n_clusters=n_clusters, method=method
    )

    if labels is not None:
        self.console.print("[green]Кластеризация завершена![/green]")
        unique, counts = np.unique(labels, return_counts=True)
        table = Table(title="Распределение по кластерам")
        table.add_column("Кластер", style="cyan")
        table.add_column("Количество фильмов", style="green")

        for cluster, count in zip(unique, counts):
            table.add_row(str(cluster), str(count))

        self.console.print(table)

    input("Нажмите Enter, чтобы продолжить...")

def pca_explain_recommendations(self):
    """Объяснение PCA рекомендаций для конкретного фильма"""
    movie_id_str = Prompt.ask("Введите ID фильма для объяснения")

    if not movie_id_str.isdigit():
        self.console.print("[red]Некорректный ID фильма[/red]")
        input("Нажмите Enter, чтобы продолжить...")
        return

    movie_id = int(movie_id_str)

    if movie_id not in self.movies.index:
        self.console.print("[red]Фильм с таким ID не найден[/red]")
        input("Нажмите Enter, чтобы продолжить...")
        return

    explanation =
self.pca_recommender.get_recommendation_explanation(movie_id)

    movie_title = self.movies.loc[movie_id, "title"]
    self.console.print(f"\n[bold]Объяснение для фильма:[/bold]
{movie_title}")
    self.console.print(f"[bold]ID:[/bold] {movie_id}")
    self.console.print(f"\n[bold]Объяснение PCA:[/bold]")
    self.console.print(explanation)

    similar = self.pca_recommender.recommend_similar_movies(movie_id,
top_k=5)
    if similar:
        self.console.print(f"\n[bold]Похожие фильмы:[/bold]")
        for sim_movie_id, score in similar:
            sim_title = self.movies.loc[sim_movie_id, "title"]
            self.console.print(f" - {sim_title} (схожесть: {score:.3f})")

    input("\nНажмите Enter, чтобы продолжить...")

if __name__ == "__main__":
    cinema = MovieLensCinema(r"../ml-latest-small",
enable_markov_visualization=False)
    cinema.menu()

```

Приложение Б

Код файла MarkovChain.py

Листинг Б – Код файла MarkovChain.py

```
import numpy as np
import os
import shutil
import networkx as nx
import matplotlib.pyplot as plt
from datetime import datetime
from rich.table import Table

class MarkovChainRecommender:
    """Внутренний класс для рекомендаций на основе цепи Маркова"""

    def __init__(self, cinema, auto_visualize=True):
        self.cinema = cinema
        self.transition_matrix = None
        self.genre_states = []
        self.auto_visualize = auto_visualize

        self._clean_markov_directory()

    def _clean_markov_directory(self):
        """Очистка папки markov при запуске программы"""
        try:
            if os.path.exists("markov"):
                shutil.rmtree("markov")
            os.makedirs("markov", exist_ok=True)
        except Exception as e:
            pass

    def reset_matrix(self):
        """Сброс матрицы переходов (вызывается при изменении оценок)"""
        self.transition_matrix = None
        self.genre_states = []

    def build_transition_matrix(self):
        """Построение матрицы переходов между жанрами с корректной нормализацией
        (вариант 1 – честная модель)"""
        if self.cinema.rates.empty:
            return False

        rated_movies = self.cinema.movies.loc[self.cinema.rates.index]

        all_genres = set()
        for genres in rated_movies["genres"]:
            all_genres.update(genres)
        self.genre_states = sorted(all_genres)

        if not self.genre_states:
            return False

        n_genres = len(self.genre_states)
        self.transition_matrix = np.zeros((n_genres, n_genres))
        state_to_idx = {genre: idx for idx, genre in
            enumerate(self.genre_states)}

        rated_indices = list(rated_movies.index)
```

```
    if len(rated_indices) < 2:
        return False

    for i in range(len(rated_indices) - 1):
        current_movie_id = rated_indices[i]
        next_movie_id = rated_indices[i + 1]

        current_genres = rated_movies.loc[current_movie_id, "genres"]
        next_genres = rated_movies.loc[next_movie_id, "genres"]

        if not current_genres or not next_genres:
            continue

        weight = 1.0 / len(next_genres)

        for curr_genre in current_genres:
            if curr_genre not in state_to_idx:
                continue
            curr_idx = state_to_idx[curr_genre]

            for next_genre in next_genres:
                if next_genre not in state_to_idx:
                    continue
                next_idx = state_to_idx[next_genre]
                self.transition_matrix[curr_idx, next_idx] += weight

    row_sums = self.transition_matrix.sum(axis=1)

    for i in range(n_genres):
        if row_sums[i] > 0:
            self.transition_matrix[i] /= row_sums[i]
        else:
            self.transition_matrix[i, i] = 1.0

    if self.auto_visualize:
        self._auto_visualize_markov_chain()
    return True

def get_genre_probabilities(self, steps=1):
    """Получение вероятностей жанров через steps шагов"""
    if self.transition_matrix is None:
        success = self.build_transition_matrix()
        if not success:
            return None

    if self.cinema.rates.empty:
        return None

    rated_movies = self.cinema.movies.loc[self.cinema.rates.index]
    last_movie_id = rated_movies.index[-1]
    last_genres = rated_movies.loc[last_movie_id, "genres"]

    initial_vector = np.zeros(len(self.genre_states))
    genre_count = len(last_genres)

    if genre_count == 0:
        return None

    for genre in last_genres:
        if genre in self.genre_states:
```

```
        idx = self.genre_states.index(genre)
        initial_vector[idx] = 1.0 / genre_count

    current_probs = initial_vector
    for _ in range(steps):
        current_probs = current_probs @ self.transition_matrix

    return dict(zip(self.genre_states, current_probs))

def recommend_movies(self, steps=1, top_k=20, min_probability=0.01):
    """Рекомендация фильмов на основе цепи Маркова"""
    genre_probs = self.get_genre_probabilities(steps)
    if not genre_probs:
        unrated_movies = self.cinema.movies[
            ~self.cinema.movies.index.isin(self.cinema.rates.index)
        ]
        return unrated_movies.sample(min(20, len(unrated_movies)))

    significant_genres = [
        genre for genre, prob in genre_probs.items() if prob >=
min_probability
    ]

    if not significant_genres:
        significant_genres = [max(genre_probs, key=genre_probs.get)]

    candidate_movies = self.cinema.movies[
        ~self.cinema.movies.index.isin(self.cinema.rates.index)
    ].copy()

    if candidate_movies.empty:
        return candidate_movies

    def calculate_score(genres):
        return sum(genre_probs.get(genre, 0) for genre in genres)

    candidate_movies["markov_score"] = candidate_movies["genres"].apply(
        calculate_score
    )

    recommendations = (
        candidate_movies[candidate_movies["markov_score"] > 0]
        .sort_values("markov_score", ascending=False)
        .head(top_k)
    )

    return recommendations.drop(columns=["markov_score"], errors="ignore")

def show_transition_matrix(self):
    """Отображение матрицы переходов"""
    if self.transition_matrix is None:
        success = self.build_transition_matrix()
        if not success:
            self.cinema.console.print(
                "[red]Не удалось построить матрицу переходов[/red]"
            )
        return

    table = Table(title="Матрица переходов между жанрами")
    table.add_column("From/To", style="cyan")
```

```
for genre in self.genre_states:
    table.add_column(genre, style="green", width=10)

for i, from_genre in enumerate(self.genre_states):
    row = [from_genre]
    for j, to_genre in enumerate(self.genre_states):
        prob = self.transition_matrix[i, j]
        if prob > 0:
            row.append(f"{prob:.3f}")
        else:
            row.append("0")
    table.add_row(*row)

self.cinema.console.print(table)

def _auto_visualize_markov_chain(self, steps=1):
    """Автоматическое построение и сохранение графа цепи Маркова с
    финальными настройками"""
    os.makedirs("markov", exist_ok=True)
    if self.transition_matrix is None or len(self.genre_states) == 0:
        return

    G = nx.DiGraph()

    for genre in self.genre_states:
        G.add_node(genre)

    edge_labels = {}

    for i, from_genre in enumerate(self.genre_states):
        for j, to_genre in enumerate(self.genre_states):
            prob = self.transition_matrix[i, j]
            if prob >= 0.001:
                G.add_edge(from_genre, to_genre, weight=prob)
                edge_labels[(from_genre, to_genre)] = f"{prob:.2f}"

    plt.figure(figsize=(18, 14), dpi=120)

    if len(self.genre_states) <= 6:
        pos = nx.circular_layout(G, scale=3.0)
    elif len(self.genre_states) <= 10:
        pos = nx.circular_layout(G, scale=3.5)
    else:
        pos = nx.spring_layout(G, k=4, iterations=150, scale=3)

    node_size = 8000
    nx.draw_networkx_nodes(
        G,
        pos,
        node_size=node_size,
        node_color="lightblue",
        alpha=0.95,
        edgecolors="navy",
        linewidths=3,
        node_shape="o",
    )

    nx.draw_networkx_labels(
        G,
        pos,
        font_size=16,
```

```

        font_weight="bold",
        font_family="DejaVu Sans",
        verticalalignment="center",
        horizontalalignment="center",
    )

    if G.edges():
        edge_weights = [G[u][v]["weight"] for u, v in G.edges()]
        max_weight = max(edge_weights) if edge_weights else 1

        edge_widths = [1.5 + (w / max_weight) * 2.5 for w in edge_weights]
        edge_alphas = [0.4 + (w / max_weight) * 0.6 for w in edge_weights]

        edges = nx.draw_networkx_edges(
            G,
            pos,
            edge_color="darkred",
            width=edge_widths,
            alpha=edge_alphas,
            arrows=True,
            arrowsize=30,
            arrowstyle="->",
            connectionstyle="arc3,rad=0.25",
            min_source_margin=20,
            min_target_margin=25,
            node_size=node_size,
            ax=plt.gca(),
        )

        nx.draw_networkx_edge_labels(
            G,
            pos,
            edge_labels=edge_labels,
            font_size=12,
            font_weight="bold",
            font_family="DejaVu Sans",
            label_pos=0.18,
            verticalalignment="center",
            horizontalalignment="center",
            bbox=dict(
                boxstyle="round,pad=0.3",
                facecolor="white",
                alpha=0.9,
                edgecolor="lightgray",
                linewidth=1.5,
            ),
        )

    title_info = f"Цепь Маркова: переходы между жанрами (k={steps})\n"
    title_info += f"Узлы: {len(G.nodes())} | Переходы: {len(G.edges())} | "
    title_info += f"Сгенерировано: {datetime.now().strftime('%d.%m.%Y %H:%M')}"

    plt.title(
        title_info,
        fontsize=18,
        fontweight="bold",
        pad=30,
        loc="center",
        fontfamily="DejaVu Sans",
    )

```

Окончание Листинга Б

```
plt.axis("off")
plt.tight_layout(pad=4.0)

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
filename = f"markov/markov_chain_k{steps}_{timestamp}.png"
plt.savefig(
    filename, dpi=150, bbox_inches="tight", facecolor="white",
    edgecolor="none"
)
plt.close()
```

Приложение В

Код файла ContinuousMarkovChain.py

Листинг В – Код файла ContinuousMarkovChain.py

```
import numpy as np
import pandas as pd
from scipy.linalg import expm
from rich.table import Table
import os
import matplotlib.pyplot as plt
import networkx as nx
from datetime import datetime, timedelta

class ContinuousMarkovRecommender:
    """Исправленная реализация непрерывной цепи Маркова с реальными временными метками"""

    def __init__(self, cinema, auto_visualize=True):
        self.cinema = cinema
        self.intensity_matrix = None
        self.genre_states = []
        self.auto_visualize = auto_visualize
        self.time_unit = 1.0

    def reset_matrix(self):
        """Сброс матрицы интенсивностей"""
        self.intensity_matrix = None
        self.genre_states = []

    def build_intensity_matrix(self):
        """Построение матрицы интенсивностей с использованием реальных данных"""
        if self.cinema.rates.empty or len(self.cinema.rates) < 2:
            return False

        ratings_chronology = self.get_ratings_chronology()
        if len(ratings_chronology) < 2:
            return False

        all_genres = set()
        for movie_id, rating, time_info in ratings_chronology:
            movie_genres = self.cinema.movies.loc[movie_id, "genres"]
            all_genres.update(movie_genres)
        self.genre_states = sorted(all_genres)

        if not self.genre_states:
            return False

        n_genres = len(self.genre_states)
        state_to_idx = {genre: idx for idx, genre in
            enumerate(self.genre_states)}

        time_in_state = {genre: 0.0 for genre in self.genre_states}
        transition_counts = np.zeros((n_genres, n_genres))

        for i in range(len(ratings_chronology) - 1):
            current_movie_id, current_rating, current_time =
            ratings_chronology[i]
            next_movie_id, next_rating, next_time = ratings_chronology[i + 1]

            current_genres = self.cinema.movies.loc[current_movie_id, "genres"]
```

```

        next_genres = self.cinema.movies.loc[next_movie_id, "genres"]

        if not current_genres or not next_genres:
            continue

        time_interval = self.calculate_time_interval(current_time,
next_time)

        for genre in current_genres:
            if genre in state_to_idx:
                time_in_state[genre] += time_interval / len(current_genres)

        for curr_genre in current_genres:
            if curr_genre not in state_to_idx:
                continue
            curr_idx = state_to_idx[curr_genre]

            for next_genre in next_genres:
                if next_genre not in state_to_idx:
                    continue
                next_idx = state_to_idx[next_genre]
                weight = 1.0 / (len(current_genres) * len(next_genres))
                transition_counts[curr_idx, next_idx] += weight

        self.intensity_matrix = np.zeros((n_genres, n_genres))

        for i, genre_i in enumerate(self.genre_states):
            total_time = time_in_state[genre_i]
            if total_time > 0:
                for j, genre_j in enumerate(self.genre_states):
                    if i != j and transition_counts[i, j] > 0:
                        self.intensity_matrix[i, j] = (
                            transition_counts[i, j] / total_time
                        )

                self.intensity_matrix[i, i] = -np.sum(self.intensity_matrix[i])

        symmetric_matrix = (self.intensity_matrix + self.intensity_matrix.T) / 2

        for i in range(n_genres):
            for j in range(n_genres):
                if i != j:
                    self.intensity_matrix[i, j] = max(symmetric_matrix[i, j], 0)

                self.intensity_matrix[i, i] = -np.sum(self.intensity_matrix[i])

        min_intensity = 0.01
        for i in range(n_genres):
            if abs(self.intensity_matrix[i, i]) < min_intensity:
                for j in range(n_genres):
                    if i != j:
                        self.intensity_matrix[i, j] += min_intensity / (n_genres
- 1)

                self.intensity_matrix[i, i] = -np.sum(self.intensity_matrix[i])

        if self.auto_visualize:
            self._auto_visualize_markov_chain()

        return True

    def get_ratings_chronology(self, random_intervals=True):

```

Продолжение Листинга В

```
"""Создание хронологии оценок с реалистичными случайными интервалами"""
if self.cinema.rates.empty:
    return []

ratings_list = []
current_time = datetime.now()

prev_time = current_time - timedelta(days=len(self.cinema.rates) * 2)

for i, (movie_id, row) in enumerate(self.cinema.rates.iterrows()):
    if random_intervals:
        interval_days = np.random.exponential(scale=2.0)
    else:
        interval_days = 1.0

    timestamp = prev_time + timedelta(days=interval_days)
    ratings_list.append((movie_id, row["rating"], timestamp))
    prev_time = timestamp

return sorted(ratings_list, key=lambda x: x[2])

def calculate_time_interval(self, time1, time2):
    """Вычисление временного интервала между двумя оценками"""
    time_diff = abs((time2 - time1).total_seconds()) / (24 * 3600)
    return max(time_diff, 0.1)

def get_genre_probabilities_continuous(self, time_t):
    """Получение вероятностей жанров в момент времени t с нормализацией"""
    if self.intensity_matrix is None:
        success = self.build_intensity_matrix()
        if not success:
            return None

    if self.cinema.rates.empty:
        return None

    ratings_chronology = self.get_ratings_chronology()
    if not ratings_chronology:
        return None

    last_movie_id, last_rating, last_time = ratings_chronology[-1]
    last_genres = self.cinema.movies.loc[last_movie_id, "genres"]

    initial_vector = np.zeros(len(self.genre_states))
    genre_count = len(last_genres)

    if genre_count == 0:
        return None

    for genre in last_genres:
        if genre in self.genre_states:
            idx = self.genre_states.index(genre)
            initial_vector[idx] = 1.0 / genre_count

    try:
        transition_matrix = expm(self.intensity_matrix * time_t)
        current_probs = initial_vector @ transition_matrix

        prob_sum = np.sum(current_probs)
        if prob_sum > 0:
            current_probs = current_probs / prob_sum
```

```

        else:
            current_probs = np.ones_like(current_probs) / len(current_probs)

    except Exception as e:
        print(f"Ошибка вычисления матричной экспоненты: {e}")
        return None

    return dict(zip(self.genre_states, current_probs))

def recommend_movies_continuous(self, time_t=1.0, top_k=20):
    """Улучшенные рекомендации, чувствительные к времени"""
    genre_probs = self.get_genre_probabilities_continuous(time_t)
    if not genre_probs:
        unrated_movies = self.cinema.movies[
            ~self.cinema.movies.index.isin(self.cinema.rates.index)
        ]
        return unrated_movies.sample(min(top_k, len(unrated_movies)))

    candidate_movies = self.cinema.movies[
        ~self.cinema.movies.index.isin(self.cinema.rates.index)
    ].copy()

    if candidate_movies.empty:
        return candidate_movies

    def calculate_score(genres, time_factor=time_t):
        if not genres:
            return 0

        base_score = sum(genre_probs.get(genre, 0) for genre in genres)

        diversity_bonus = len(set(genres) & set(genre_probs.keys())) /
len(genres)

        time_weight = min(time_t / 5.0, 1.0)
        final_score = base_score * (1 - time_weight) + diversity_bonus *
time_weight

        return final_score

    candidate_movies["continuous_score"] = candidate_movies["genres"].apply(
        lambda genres: calculate_score(genres, time_t)
    )

    max_score = candidate_movies["continuous_score"].max()
    if max_score > 0:
        candidate_movies["continuous_score"] = (
            candidate_movies["continuous_score"] / max_score
        )

    recommendations = (
        candidate_movies[candidate_movies["continuous_score"] > 0.01]
        .sort_values("continuous_score", ascending=False)
        .head(top_k)
    )

    return recommendations.drop(columns=["continuous_score"],
errors="ignore")

def show_intensity_matrix(self):
    """Отображение матрицы интенсивностей с дополнительной информацией"""

```

```
        if self.intensity_matrix is None:
            success = self.build_intensity_matrix()
            if not success:
                self.cinema.console.print(
                    "[red]Не удалось построить матрицу интенсивностей[/red]"
                )
            return

        table = Table(
            title="Матрица интенсивностей переходов (непрерывная цепь Маркова)"
        )
        table.add_column("From/To", style="cyan")

        for genre in self.genre_states:
            table.add_column(genre, style="green", width=10)

        for i, from_genre in enumerate(self.genre_states):
            row = [from_genre]
            for j, to_genre in enumerate(self.genre_states):
                intensity = self.intensity_matrix[i, j]
                if abs(intensity) > 0.001:
                    if i == j:
                        row.append(f"[red]{intensity:+.3f}[/red]")
                    else:
                        row.append(f"{intensity:+.3f}")
                else:
                    row.append("0.000")
            table.add_row(*row)

        self.cinema.console.print(table)

        self.cinema.console.print(
            f"\n[bold]Размер матрицы:[/bold] {len(self.genre_states)}x{len(self.genre_states)}"
        )
        self.cinema.console.print(
            f"[bold]Количество жанров:[/bold] {len(self.genre_states)}"
        )

    def _auto_visualize_markov_chain(self, time_t=1.0):
        """Автоматическая визуализация и сохранение графа непрерывной цепи
        Маркова"""
        os.makedirs("markov", exist_ok=True)
        if self.intensity_matrix is None or len(self.genre_states) == 0:
            return

        G = nx.DiGraph()

        for genre in self.genre_states:
            G.add_node(genre)

        edge_labels = {}

        for i, from_genre in enumerate(self.genre_states):
            for j, to_genre in enumerate(self.genre_states):
                intensity = self.intensity_matrix[i, j]
                if intensity > 0.001:
                    G.add_edge(from_genre, to_genre, weight=intensity)
                    edge_labels[(from_genre, to_genre)] = f"{intensity:.2f}"

        plt.figure(figsize=(18, 14), dpi=120)
```

```
if len(self.genre_states) <= 6:
    pos = nx.circular_layout(G, scale=3.0)
elif len(self.genre_states) <= 10:
    pos = nx.circular_layout(G, scale=3.5)
else:
    pos = nx.spring_layout(G, k=4, iterations=150, scale=3)

node_size = 8000
nx.draw_networkx_nodes(
    G,
    pos,
    node_size=node_size,
    node_color="lightblue",
    alpha=0.95,
    edgecolors="navy",
    linewidths=3,
    node_shape="o",
)

nx.draw_networkx_labels(
    G,
    pos,
    font_size=16,
    font_weight="bold",
    font_family="DejaVu Sans",
    verticalalignment="center",
    horizontalalignment="center",
)

if G.edges():
    edge_weights = [G[u][v]["weight"] for u, v in G.edges()]
    max_weight = max(edge_weights) if edge_weights else 1

    edge_widths = [1.5 + (w / max_weight) * 2.5 for w in edge_weights]
    edge_alphas = [0.4 + (w / max_weight) * 0.6 for w in edge_weights]

    edges = nx.draw_networkx_edges(
        G,
        pos,
        edge_color="darkred",
        width=edge_widths,
        alpha=edge_alphas,
        arrows=True,
        arrowsize=30,
        arrowstyle="-|>",
        connectionstyle="arc3,rad=0.25",
        min_source_margin=20,
        min_target_margin=25,
        node_size=node_size,
        ax=plt.gca(),
    )

    nx.draw_networkx_edge_labels(
        G,
        pos,
        edge_labels=edge_labels,
        font_size=12,
        font_weight="bold",
        font_family="DejaVu Sans",
        label_pos=0.18,
```

Окончание Листинга В

```
        verticalalignment="center",
        horizontalalignment="center",
        bbox=dict(
            boxstyle="round,pad=0.3",
            facecolor="white",
            alpha=0.9,
            edgecolor="lightgray",
            linewidth=1.5,
        ),
    ),

    title_info = f"Непрерывная цепь Маркова: интенсивности переходов
(t={time_t})\n"
    title_info += f"Узлы: {len(G.nodes())} | Переходы: {len(G.edges())} | "
    title_info += f"Сгенерировано: {datetime.now().strftime('%d.%m.%Y
%H:%M')}"

    plt.title(
        title_info,
        fontsize=18,
        fontweight="bold",
        pad=30,
        loc="center",
        fontfamily="DejaVu Sans",
    )

    plt.axis("off")
    plt.tight_layout(pad=4.0)

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f"markov/continuous_markov_chain_t{time_t}_{timestamp}.png"
    plt.savefig(
        filename, dpi=150, bbox_inches="tight", facecolor="white",
edgecolor="none"
    )
    plt.close()
```

Приложение Г

Код файла SVDRecommender.py

Листинг Г – Код файла SVDRecommender.py

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import (
    mean_squared_error,
    mean_absolute_error,
    root_mean_squared_error,
)
from sklearn.decomposition import PCA
from sklearn.mixture import GaussianMixture
from sklearn.cluster import AgglomerativeClustering

class SVDRecommender:
    def __init__(self, cinema, k=20):
        self.cinema = cinema
        self.k = k
        self.U = None
        self.D = None
        self.Vt = None
        self.pred_matrix = None

    def build(self):
        """SVD по матрице рейтингов"""
        R = self.cinema.get_rating_matrix()

        R_filled = R.fillna(0).values

        U, D, Vt = np.linalg.svd(R_filled, full_matrices=False)

        U_k = U[:, : self.k]
        D_k = np.diag(D[: self.k])
        Vt_k = Vt[:, : self.k, :]

        self.U = U_k
        self.D = D_k
        self.Vt = Vt_k

        self.pred_matrix = np.dot(U_k, np.dot(D_k, Vt_k))

        return self.pred_matrix

    def evaluate(self):
        """Вычисление RMSE, MSE, MAE по реальным оценкам"""
        ratings = self.cinema.ratings.reset_index()
        movie_ids = self.cinema.get_rating_matrix().columns.tolist()
        user_ids = self.cinema.get_rating_matrix().index.tolist()

        preds = []
        reals = []

        for _, row in ratings.iterrows():
            uid = row["userId"]
            mid = row["movieId"]

            if uid not in user_ids or mid not in movie_ids:
                continue
```

```

        u_idx = user_ids.index(uid)
        m_idx = movie_ids.index(mid)
        pred = self.pred_matrix[u_idx, m_idx]

        preds.append(pred)
        reals.append(row["rating"])

    mse = mean_squared_error(reals, preds)
    rmse = root_mean_squared_error(reals, preds)
    mae = mean_absolute_error(reals, preds)

    return mse, rmse, mae

def recommend_for_current_user(self, top_k=20):
    """Рекомендации для текущего пользователя"""
    R = self.cinema.get_rating_matrix()
    movie_ids = R.columns.tolist()

    current_user_id = -1
    current_ratings = pd.Series([np.nan] * len(movie_ids), index=movie_ids)

    for mid in self.cinema.rates.index:
        if mid in movie_ids:
            current_ratings[mid] = self.cinema.rates.loc[mid, "rating"]

    R_ext = pd.concat(
        [R, pd.DataFrame([current_ratings], index=[current_user_id])]
    ).fillna(0)

    u_vec = np.dot(
        np.dot(R_ext.loc[current_user_id].values, self.Vt.T),
        np.linalg.inv(self.D)
    )

    preds = np.dot(u_vec, self.Vt)

    result = pd.DataFrame({"movieId": movie_ids, "pred": preds})

    result = result[~result["movieId"].isin(self.cinema.rates.index)]

    result = result.sort_values("pred", ascending=False).head(top_k)

    return self.cinema.movies.loc[result["movieId"]]

def evaluate_over_k(self, k_min=2, k_max=50):
    R = self.cinema.get_rating_matrix()
    R_filled = R.fillna(0).values
    ratings = self.cinema.ratings.reset_index()

    movie_ids = R.columns.tolist()
    user_ids = R.index.tolist()

    ks = []
    mses = []
    rmses = []
    maes = []

    for k in range(k_min, k_max + 1):
        self.cinema.console.print(f"Построение SVD для k={k}: ",
                                highlight=False, end="")

```

```

        U, D, Vt = np.linalg.svd(R_filled, full_matrices=False)

        U_k = U[:, :k]
        D_k = np.diag(D[:k])
        Vt_k = Vt[:k, :]

        pred_matrix = np.dot(U_k, np.dot(D_k, Vt_k))

        preds = []
        reals = []

        for _, row in ratings.iterrows():
            uid = row["userId"]
            mid = row["movieId"]

            if uid not in user_ids or mid not in movie_ids:
                continue

            u_idx = user_ids.index(uid)
            m_idx = movie_ids.index(mid)

            preds.append(pred_matrix[u_idx, m_idx])
            reals.append(row["rating"])

        mse = mean_squared_error(reals, preds)
        rmse = np.sqrt(mse)
        mae = mean_absolute_error(reals, preds)

        self.cinema.console.print(f"[green]MSE:[/green] {mse:.4f},
[green]RMSE:[/green] {rmse:.4f}, [green]MAE:[/green] {mae:.4f}")

        ks.append(k)
        msess.append(mse)
        rmsses.append(rmse)
        maes.append(mae)

        plt.figure(figsize=(10, 6))
        plt.plot(ks, msess, label="MSE")
        plt.plot(ks, rmsses, label="RMSE")
        plt.plot(ks, maes, label="MAE")
        plt.xlabel("Число скрытых факторов")
        plt.ylabel("Значение метрики")
        plt.title("Зависимость MSE, RMSE, MAE от числа факторов k")
        plt.legend()
        plt.grid(True)
        plt.show()

    def cluster_users(self, n_clusters=4):
        if self.U is None:
            raise ValueError("Сначала постройте SVD с помощью build().")

        U_k = self.U

        pca = PCA(n_components=2)
        U_2d = pca.fit_transform(U_k)

        model = AgglomerativeClustering(n_clusters=n_clusters)
        labels = model.fit_predict(U_2d)

        plt.figure(figsize=(8, 6))

```

Окончание Листинга Г

```
plt.scatter(U_2d[:, 0], U_2d[:, 1], c=labels, cmap="tab10")
plt.title(f"Кластеризация пользователей")
plt.xlabel("PCA component 1")
plt.ylabel("PCA component 2")
plt.grid(True)
plt.show()

return labels
```

Приложение Д

Код файла PCARecommender.py

Листинг Д – Код файла PCARecommender.py

```
import numpy as np
import pandas as pd
from datetime import datetime
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.cluster import AgglomerativeClustering
from sklearn.mixture import GaussianMixture
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error, mean_absolute_error

class PCARecommender:
    def __init__(self, cinema, n_components=None):
        self.cinema = cinema
        self.n_components = n_components
        self.scaler = StandardScaler()
        self.pca = None

        self.feature_matrix = None
        self.feature_names = None
        self.movie_ids = None

        self.X_scaled = None
        self.X_pca = None
        self.reconstructed = None
        self.explained_variance_ratio_ = None
        self.components_ = None

    def create_movie_feature_matrix(self):
        """Создание семантически осмысленной матрицы признаков фильмов"""
        movies = self.cinema.movies.copy()

        if "year" not in movies.columns:
            movies["year"] = pd.NA
        movies["year"] = movies["year"].fillna(movies["year"].median())

        all_genres = set()
        for genres_list in movies["genres"]:
            if isinstance(genres_list, list):
                all_genres.update(genres_list)

        all_genres = sorted(list(all_genres))

        for genre in all_genres:
            movies[f"genre_{genre}"] = movies["genres"].apply(
                lambda x: 1 if isinstance(x, list) and genre in x else 0
            )

        current_year = datetime.now().year
        movies["movie_age"] = (current_year -
            movies["year"]).fillna(0).astype(float)
        movies["is_old"] = (movies["movie_age"] > 30).astype(int)
        movies["is_recent"] = (movies["movie_age"] < 10).astype(int)

        if isinstance(self.cinema.ratings.index, pd.MultiIndex):
            ratings_df = self.cinema.ratings.reset_index()
```

```

else:
    ratings_df = self.cinema.ratings.copy().reset_index()

    rating_stats = (
        ratings_df.groupby("movieId")["rating"]
        .agg(["mean", "std", "count"])
        .rename(
            columns={
                "mean": "rating_mean",
                "std": "rating_std",
                "count": "rating_count",
            }
        )
    )
    rating_stats = rating_stats.fillna(0)
    movies = movies.join(rating_stats, how="left")

    movies["rating_mean"] = movies["rating_mean"].fillna(0).astype(float)
    movies["rating_std"] = movies["rating_std"].fillna(0).astype(float)
    movies["rating_count"] = movies["rating_count"].fillna(0).astype(float)

    numeric_features = ["movie_age", "rating_mean", "rating_std",
"rating_count"]
    for feature in numeric_features:
        if movies[feature].std(ddof=0) == 0:
            movies[f"{feature}_norm"] = 0.0
        else:
            movies[f"{feature}_norm"] = (
                movies[feature] - movies[feature].mean()
            ) / movies[feature].std(ddof=0)

    genre_cols = [f"genre_{g}" for g in all_genres]
    feature_cols = genre_cols + [
        "is_old",
        "is_recent",
        "movie_age_norm",
        "rating_mean_norm",
        "rating_std_norm",
        "rating_count_norm",
    ]

    for col in feature_cols:
        if col not in movies.columns:
            movies[col] = 0.0

    self.feature_matrix =
movies[feature_cols].fillna(0).values.astype(float)
    self.feature_names = feature_cols
    self.movie_ids = movies.index.tolist()

    return self.feature_matrix

def build_pca_model(self):
    """Построение PCA модели"""
    try:
        X = self.create_movie_feature_matrix()
        if X is None or X.size == 0:
            raise ValueError("Пустая матрица признаков фильмов")

        self.X_scaled = self.scaler.fit_transform(X)

```

```
        self.pca = PCA(n_components=self.n_components)
        self.X_pca = self.pca.fit_transform(self.X_scaled)

        X_reconstructed_scaled = self.pca.inverse_transform(self.X_pca)
        self.reconstructed =
self.scaler.inverse_transform(X_reconstructed_scaled)

        self.explained_variance_ratio_ = self.pca.explained_variance_ratio_
        self.components_ = self.pca.components_

        return True
    except Exception as e:
        print(f"Ошибка построения PCA: {e}")
        return False

def evaluate_reconstruction(self):
    """Оценка качества восстановления"""
    if self.reconstructed is None:
        return None, None, None

    X_original = self.create_movie_feature_matrix()
    mse = mean_squared_error(X_original.flatten(),
self.reconstructed.flatten())
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(X_original.flatten(),
self.reconstructed.flatten())
    return mse, rmse, mae

def recommend_similar_movies(self, movie_id, top_k=10):
    """Рекомендация похожих фильмов"""
    if self.pca is None or self.X_pca is None:
        if not self.build_pca_model():
            return []

    try:
        idx = self.movie_ids.index(movie_id)
    except ValueError:
        return []

    target_vec = self.X_pca[idx].reshape(1, -1)
    sims = cosine_similarity(target_vec, self.X_pca)[0]
    pairs = [
        (mid, float(sim))
        for mid, sim in zip(self.movie_ids, sims)
        if mid != movie_id
    ]
    pairs.sort(key=lambda x: x[1], reverse=True)
    return pairs[:top_k]

def recommend_for_current_user(self, top_k=20):
    """Рекомендации для текущего пользователя на основе PCA"""
    if self.pca is None or self.X_pca is None:
        if not self.build_pca_model():
            return pd.DataFrame(columns=["movieId", "score"])

    rated = self.cinema.rates.copy()
    if rated.empty:
        return pd.DataFrame(columns=["movieId", "score"])

    movie_idx_map = {mid: i for i, mid in enumerate(self.movie_ids)}
    user_vec = None
```

```

weights_sum = 0.0

for mid, row in rated.iterrows():
    if mid not in movie_idx_map:
        continue
    idx = movie_idx_map[mid]
    rating = float(row["rating"])
    vec = self.X_pca[idx]
    if user_vec is None:
        user_vec = rating * vec
    else:
        user_vec += rating * vec
    weights_sum += abs(rating)

if user_vec is None or weights_sum == 0:
    return pd.DataFrame(columns=["movieId", "score"])

user_vec = (user_vec / weights_sum).reshape(1, -1)

sims = cosine_similarity(user_vec, self.X_pca)[0]
results = []
for mid, sim in zip(self.movie_ids, sims):
    if mid in rated.index:
        continue
    results.append((mid, float(sim)))

results.sort(key=lambda x: x[1], reverse=True)
top = results[:top_k]

df = pd.DataFrame(top, columns=["movieId", "score"])
movies_meta = self.cinema.movies.loc[df["movieId"]].reset_index()
df = df.merge(
    movies_meta[["movieId", "title", "genres"]], on="movieId",
how="left"
)

return df

def get_component_analysis(self, component_idx, top_n=20):
    """Анализ компоненты"""
    if self.pca is None or self.components_ is None:
        return None

    if component_idx < 0 or component_idx >= self.components_.shape[0]:
        return None

    comp = self.components_[component_idx]
    feat_importance = list(zip(self.feature_names, comp))
    feat_importance.sort(key=lambda x: abs(x[1]), reverse=True)
    return feat_importance[:top_n]

def plot_variance_explained(self):
    """График объясненной дисперсии"""
    if self.pca is None or self.explained_variance_ratio_ is None:
        print("PCA модель ещё не построена")
        return

    cum_var = np.cumsum(self.explained_variance_ratio_)
    plt.figure(figsize=(8, 5))
    plt.plot(np.arange(1, len(cum_var) + 1), cum_var, marker="o")
    plt.xlabel("Число компонент")

```

```
plt.ylabel("Объясненная дисперсия")
plt.title("PCA: объясненная дисперсия")
plt.grid(True)
plt.show()

def cluster_movies(self, n_clusters=6, method="agglomerative"):
    """Классификация фильмов в PCA-пространстве"""
    if self.pca is None or self.X_pca is None:
        if not self.build_pca_model():
            return None

    if method == "gmm":
        model = GaussianMixture(n_components=n_clusters, random_state=42)
        labels = model.fit_predict(self.X_pca)
    else:
        model = AgglomerativeClustering(n_clusters=n_clusters)
        labels = model.fit_predict(self.X_pca)

    plt.figure(figsize=(8, 6))
    if self.X_pca.shape[1] >= 2:
        plt.scatter(
            self.X_pca[:, 0], self.X_pca[:, 1], c=labels, cmap="tab10", s=10
        )
        plt.title(f"Movie clusters (method={method})")
        plt.xlabel("PC1")
        plt.ylabel("PC2")
        plt.grid(True)
        plt.show()

    return labels

def get_recommendation_explanation(self, movie_id, top_features=5):
    """Объяснение рекомендации для фильма"""
    if self.pca is None:
        return "Модель PCA не построена"

    try:
        movie_idx = self.movie_ids.index(movie_id)
        movie_features = self.feature_matrix[movie_idx]

        feature_importance = []
        for i, (name, value) in enumerate(zip(self.feature_names,
movie_features)):
            if value != 0:
                feature_importance.append((name, value))

        feature_importance.sort(key=lambda x: abs(x[1]), reverse=True)

        explanation = f"Фильм характеризуется: "
        features_desc = []
        for name, value in feature_importance[:top_features]:
            if "genre_" in name:
                genre = name.replace("genre_", "")
                features_desc.append(f"жанр '{genre}'")
            elif "rating_mean" in name:
                features_desc.append(
                    f"высоким средним рейтингом"
                    if value > 0
                    else f"низким средним рейтингом"
                )
            elif "is_old" in name and value > 0:
```

Окончание Листинга Д

```
        features_desc.append("старым релизом")
    elif "is_recent" in name and value > 0:
        features_desc.append("новым релизом")

    explanation += ", ".join(features_desc)
    return explanation

except ValueError:
    return "Фильм не найден в базе"
```